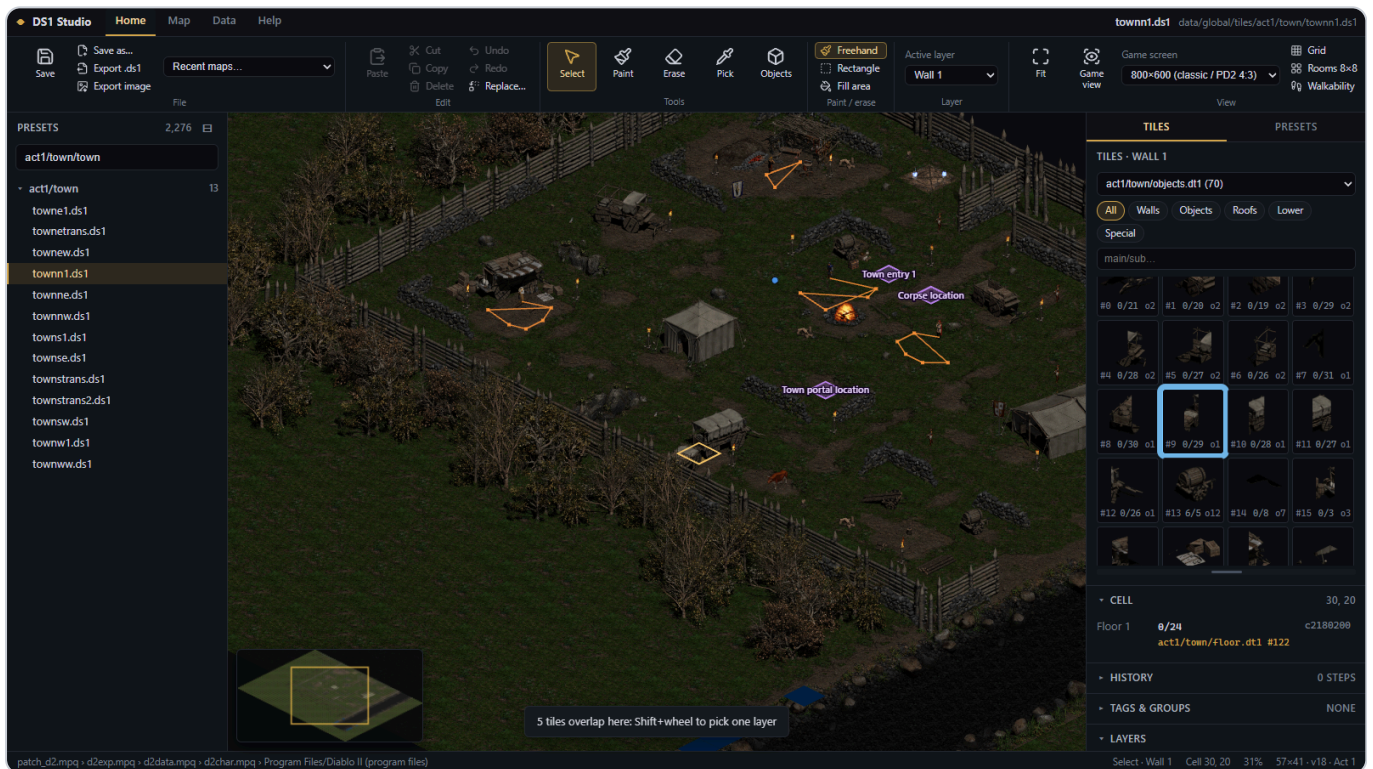




DS1 Studio

How-To Manual — the map editor for classic Diablo II (1.13 / 1.14) and its mods



Covers DS1 Studio 0.1.13 · Windows and Linux · github.com/RoofooEvazan/ds1-studio

Every blue link in this document jumps to that part of the manual. Use [Contents](#) to find your way, and the [↑ Contents](#) link at the top of every chapter to come back.

Contents

1 Getting started

- 1.1 [Installing](#)
- 1.2 [First start: choosing your folders](#)
- 1.3 [A tour of the window](#)
- 1.4 [Opening a map](#)
- 1.5 [Saving, and where files go](#)
- 1.6 [Autosave and recovery](#)
- 1.7 [Recent maps](#)
- 1.8 [Updates](#)

2 Viewing maps

- 2.1 [Moving around and zooming](#)
- 2.2 [The minimap](#)
- 2.3 [Layers: showing and hiding](#)
- 2.4 [View modes, grid, rooms](#)
- 2.5 [Game view](#)
- 2.6 [Objects, sprites and animation](#)
- 2.7 [Special tiles](#)
- 2.8 [Overlapping \(stacked\) tiles](#)

3 Editing tiles

- 3.1 [The tools](#)
- 3.2 [The Tiles panel](#)
- 3.3 [Painting: freehand, rectangle, fill](#)
- 3.4 [Selecting, copying and moving](#)
- 3.5 [Find & replace, re-roll](#)
- 3.6 [Undo and the History panel](#)
- 3.7 [The Cell panel](#)
- 3.8 [Walkability: sub-tile flags](#)
- 3.9 [Resizing and new maps](#)

4 Objects and NPCs

- 4.1 [Placing from the gallery](#)
- 4.2 [Moving and editing objects](#)
- 4.3 ["In this map": find any object](#)

5 Presets

6 Warps: connecting maps

7 Importing DS1 and DT1 files

- 7.1 [Import DS1 \(with its tile libraries\)](#)
- 7.2 [The DT1 library: import, browse, build](#)

8 Tile libraries and the DT1 editor

- 8.1 [Tile libraries](#)
- 8.2 [The DT1 editor](#)
- 8.3 [Recolouring](#)
- 8.4 [Pixel painting](#)
- 8.5 [Tile settings \(.ini\)](#)

9 Game data

- 9.1 [Data tables](#)
- 9.2 [Add to game](#)
- 9.3 [Cube recipe](#)
- 9.4 [Compatibility check](#)
- 9.5 [The automap](#)

10 Sharing and exporting

11 Help, feedback and shortcuts

12 Safety and troubleshooting

A Keyboard and mouse reference

B Glossary

1 Getting started

DS1 Studio opens the map presets (**DS1** files) of classic Diablo II and your mod straight from the game's files, draws them the way the game does, and lets you edit their tiles, objects, the game's level tables, the automap and the tile libraries (**DT1** files) themselves. See the [Glossary](#) for any term you don't know.

1.1 Installing

Download the latest version from the [Releases page](#) on GitHub:

System	File	Notes
Windows 10/11	<code>..._x64-setup.exe</code>	Recommended. Installs for your user and updates itself.
	<code>..._x64_en-US.msi</code>	For managed installs.
Linux	<code>...AppImage</code>	Runs on most distributions; make it executable and double-click. Updates itself.
	<code>...deb</code> / <code>...rpm</code>	Debian/Ubuntu, Fedora/openSUSE packages.

Note The Windows installer isn't code-signed yet, so Windows may say "Windows protected your PC". Click **More info** → **Run anyway**.

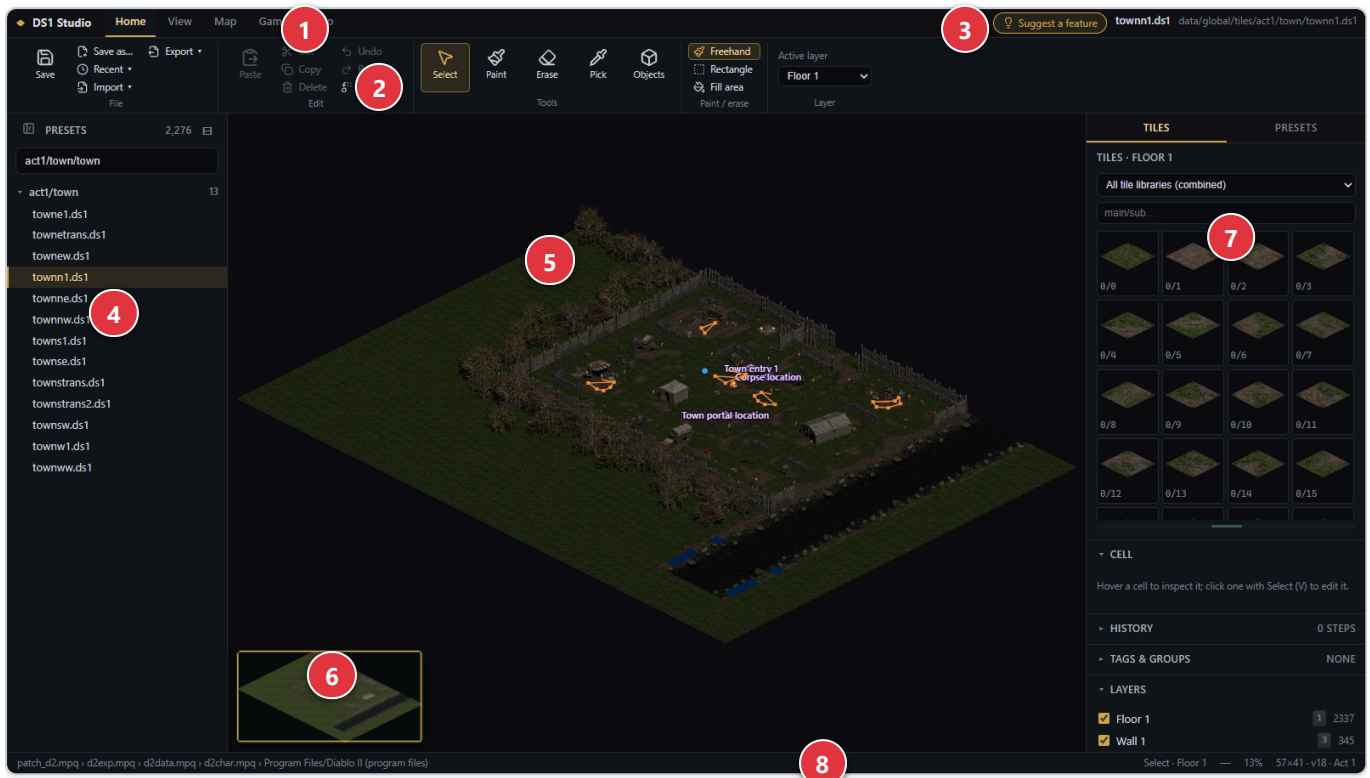
1.2 First start: choosing your folders

The first time it runs, DS1 Studio asks for two folders:

- **Diablo II** — the folder with `d2data.mpq`, `d2exp.mpq` and `patch_d2.mpq`. It is only ever *read*.
- **Mod folder** (optional) — your mod's folder with its `data/` tree. This is where everything you save goes ([see Saving](#)). Tick *Also read the mod folder's own .mpq files* if your mod ships its data in MPQs.

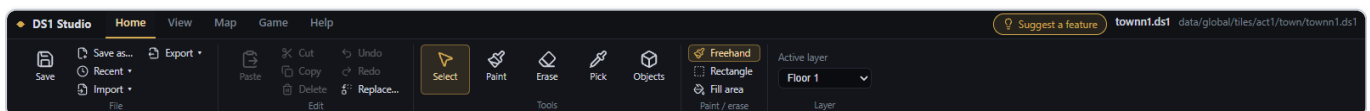
On Linux, point it at the Diablo II folder inside your Wine prefix. You can change the folders later with **Home** → **File** → **Folders...** Nothing else is needed: object names and sprites come from the game's own files, and special tiles get DS1 Studio's own markers.

1.3 A tour of the window



The main window with Act 1's Rogue Encampment open.

- 1 Tabs:** Home (files and editing), View (how the map is shown, and the view modes), Map (the map and its tile libraries), Game (the game's tables, and checks) and Help switch the ribbon below.
- 2 Ribbon:** the commands, in labelled groups (File, Edit, Tools, Paint / erase, Layer on Home). Hover any button for what it does and its shortcut.
- 3 Suggest a feature** and the open map's name and path. A dot after the name means unsaved changes.
- 4 Presets list:** every DS1 in the game and your mod. Type in the box to filter (see [Opening a map](#)). The button left of its title folds it away to a thin strip for more map room; click the strip to bring it back.
- 5 Map view:** the map as the game draws it (see [Viewing maps](#)).
- 6 Minimap:** the whole map; click it to move there (2.2).
- 7 Side panels:** Tiles/Presets, Selection, Cell, History, Layers, Map... (they change with the tool). In a view mode (Walkability, Automap, Level light, Roof hiding) they make way for that mode's own panel (see [View modes](#)).
- 8 Status bar:** the data sources, the tool and layer, the cell under the mouse, zoom, size and act.



The Home ribbon. The other tabs are described in their chapters: [View](#), [Map](#), [Game](#), [Help](#).

1.4 Opening a map

- 1** Type part of the path in the box above the presets list, e.g. `act1/town` or `arcane`.

- 2 Click a map in the list. It opens with the tile libraries the game would use for it (from `LvlPrest.txt` → `Levels.txt` → `LvlTypes.txt`) and the right act palette.
- 3 Press `F` to fit the whole map in the view.

You can also reopen a recent map (1.7), create a new one (3.9) or import one (7.1).

1.5 Saving, and where files go

Save (`Ctrl` + `S`) writes the map into your **mod folder**, under the same path it has in the game. If a file is replaced, the original is kept next to it as `<name>.bak` (only the first time, so the true original survives). **Save as...** saves under a new path. Your Diablo II install is never written to.

Everything else DS1 Studio writes — edited DT1s, `.txt` tables, presets — also goes into the mod folder with the same `.bak` safety. See [Safety and troubleshooting](#).

Switching and closing maps. Opening another map saves the one you're leaving first (**Home** → **Preferences...** turns this off; then a map with unsaved changes asks whether to save or discard them). `Esc` with nothing selected asks to close the map: **Save and close**, **Discard and close** or **Cancel**.

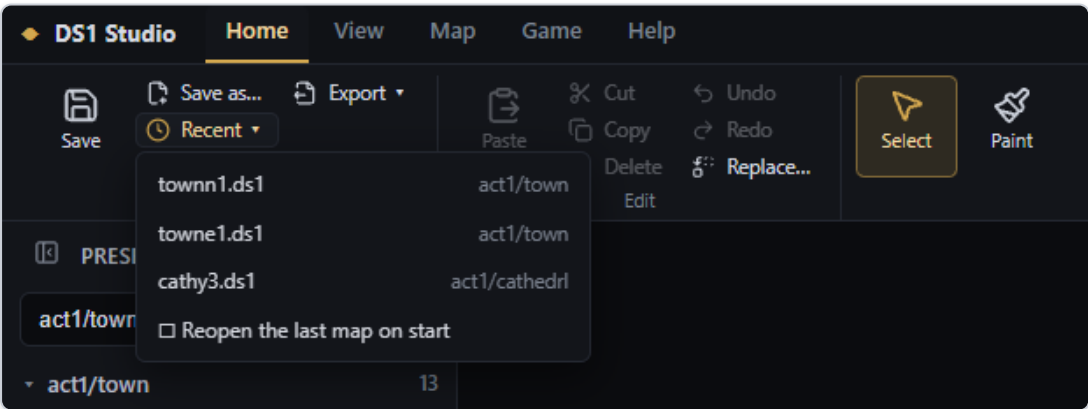
1.6 Autosave and recovery

While a map has unsaved changes, DS1 Studio keeps a copy of it every 20 seconds in its own storage (never in your game or mod folders). If the app crashes or is closed without saving:

- the start page lists **Unsaved work kept from an earlier session**, and
- opening that map offers **Restore** (the autosaved version opens, unsaved until you save) or **Discard them**.

Saving, or undoing back to the saved state, removes the kept copy.

1.7 Recent maps



Home → File → Recent.

Home → **File** → **Recent** ▾ lists the maps you opened last; the start page lists them too. Tick **Reopen the last map on start** to have your last map open automatically.

1.8 Updates

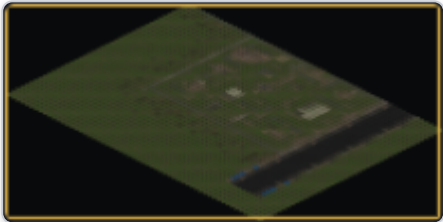
The app checks GitHub once a day and tells you when a new version is out. **Help** → **Check for updates** downloads and installs it and restarts DS1 Studio. **Help** → **About** shows the version and build.

2.1 Moving around and zooming

Zoom	Mouse wheel (zooms around the mouse)
Pan	Arrow keys (Shift = faster), Space + drag, or drag with the middle or right mouse button
Fit the whole map	F
Jump anywhere	Click or drag on the minimap

At 100% you see exactly the game's pixels; zoomed out, tiles are properly downscaled instead of turning grainy.

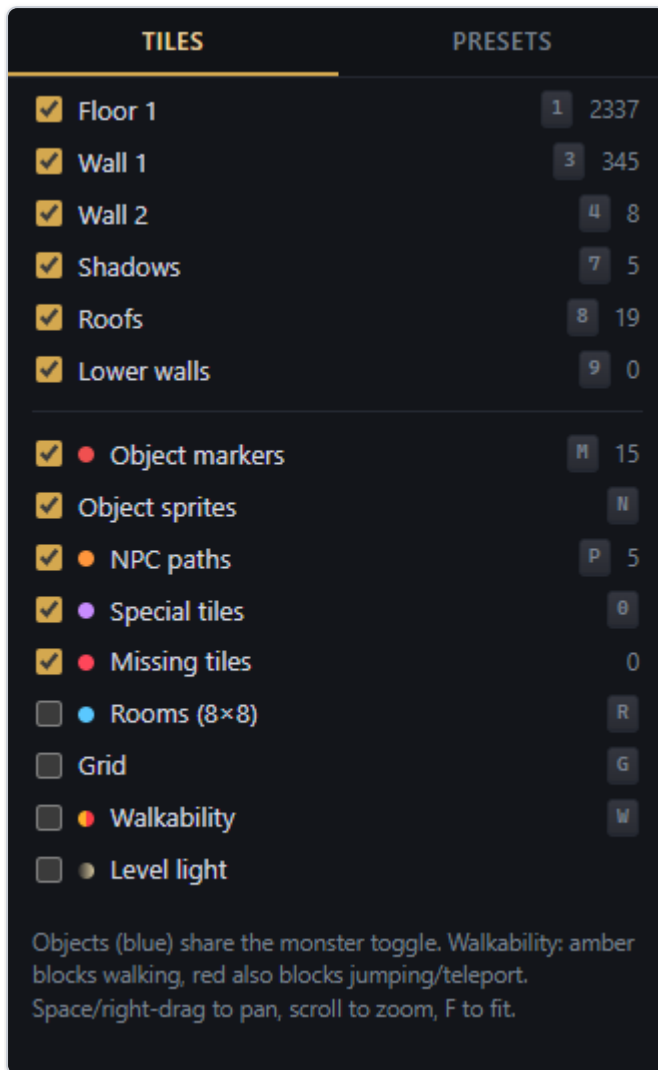
2.2 The minimap



The minimap. The outlined part is what the map view shows.

The bottom-left corner shows the whole map with the visible area outlined. Click or drag on it to move the view there. Toggle it with [K](#) or **View** → **Show** → **Minimap**.

2.3 Layers: showing and hiding



The Layers panel.

A map has up to 2 floor layers, 4 wall layers (walls, roofs, lower walls, special tiles, trees and objects that are tiles), shadows, and optionally a tag layer.

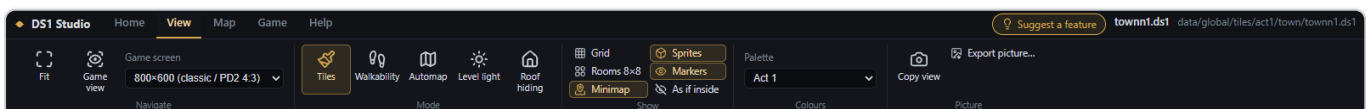
The **Layers** panel shows or hides each one and says how many cells it uses. The number keys toggle them too: **1**/**2** floors, **3**–**6** walls, **7** shadows, **8** roofs, **9** lower walls, **0** special tiles.

Hiding a layer only affects the view. To *edit* a layer, make it the active layer (3.1).

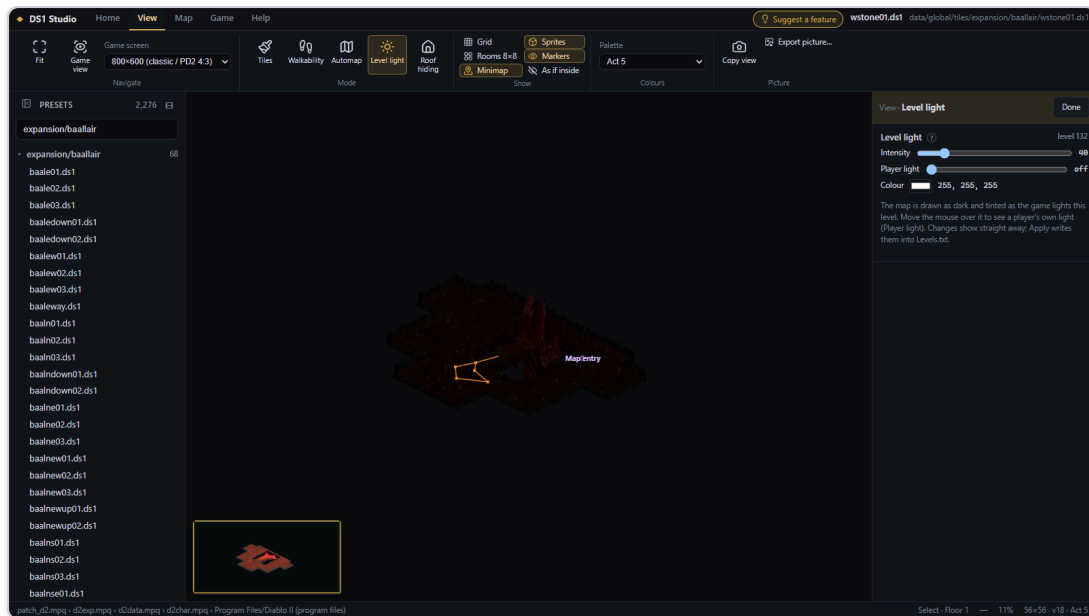
2.4 View modes, grid, rooms

Walkable area. The box at the bottom right of the map shows how much of the map a player can stand on, in tiles² (the walkable sub-tiles of the Walkability view, 25 to a tile), updated as you edit. It is grey below 3,300, green from 3,300 blending to orange at 3,800, orange blending to red by 4,300, and redder above that.

View modes. The **View** tab's **Mode** group switches what the right-hand panel is for, one mode at a time: **Tiles** (editing: the Tiles, Cell, History, Layers and Map panels), **Walkability** (**W**), **Automap** (**A**), **Level light** (**Shift** + **L**) and **Roof hiding** (**H**). Each mode draws its view on the map and replaces the side panels with only its own options; **Done** at the top of the panel (or the mode's button or key again) goes back to Tiles. **Tab** steps through the views in turn (Tiles, Walkability, Automap, Level light, Roof hiding) and **Shift** + **Tab** goes back. The **Show** group's overlays (grid, rooms, minimap, sprites, markers, As if inside) are not modes: they work in any mode.



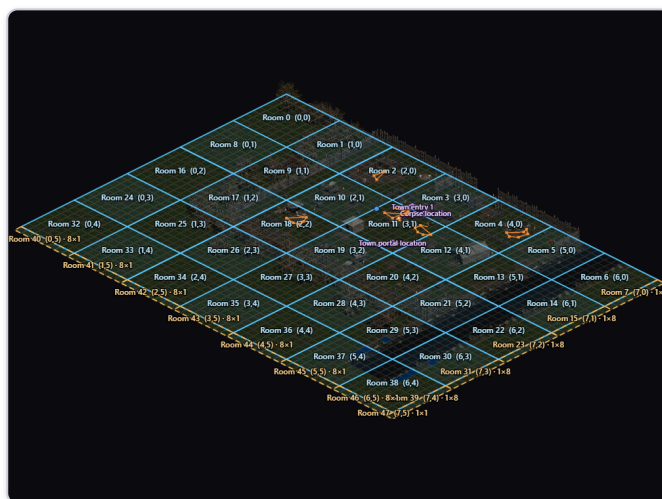
The View ribbon: Navigate, Mode (one at a time), Show (overlays), Colours and Picture.



View → Level light on the Worldstone Chamber (Intensity 40): the side panel holds only the light's settings.



Walkability (W): amber sub-tiles block walking; red also block jumping/teleporting.



Rooms 8×8 (R) and **Grid (G)**: the rooms the game builds levels from, and the cell grid.

Automap (A) draws the in-game automap over the map, as players will see it ([see The automap](#)).

Level light (View → Level light, **Shift** + **L**) draws the map as brightly and in the colour the game lights its level: Levels.txt's **Intensity** (1-255) and **Red / Green / Blue** ; 0 means the act's own daylight, as towns and outdoor levels use. The **Map** panel shows and changes them (**Level light**: slider, colour, **Apply to Levels.txt**), previewing while you move the slider. In game the player's own light radius also brightens the area around them.

2.5 Game view



Game view: the 800×600 screen around the selection.

Game view (**Z**) zooms to exactly what the character sees in game, centred on the selection, and shades everything else. Pick the screen size next to it: 640×480, 800×600, PD2's widescreen sizes, or a custom size.

2.6 Objects, sprites and animation

Objects and NPCs are drawn with their real sprites, animated at the game's speeds, with fire, glows, magic and fog blended as in game. **Sprites** (**N**) toggles the sprites, (**M**) the object markers, (**P**) the NPC paths. Animated floor tiles (water, lava) animate too. See [Objects and NPCs](#) for editing them.

2.7 Special tiles

Special tiles are invisible in game: they mark warps, where players arrive in town or on a map, the corpse and town-portal spots. DS1 Studio draws a labelled marker on each, e.g. *Town entry 1*, *Map entry*, *Corpse location*. Warps show where they lead, e.g. *Warp · link 1 → Catacombs Level 1*; see [Warps: connecting maps](#).

Placing them: right-click a cell and choose **Place a special tile here**: Map entry, Town entry 1 and 2, Corpse location, Town portal location, or a warp (Vis 0–7). It goes on the first wall layer free at that cell (a new wall layer if all are taken). They are also in the [Tiles panel](#) with a wall layer active: the **Special** filter lists them (from *builtin/special tiles*, DS1 Studio's own stand-ins, since the game draws nothing for them).

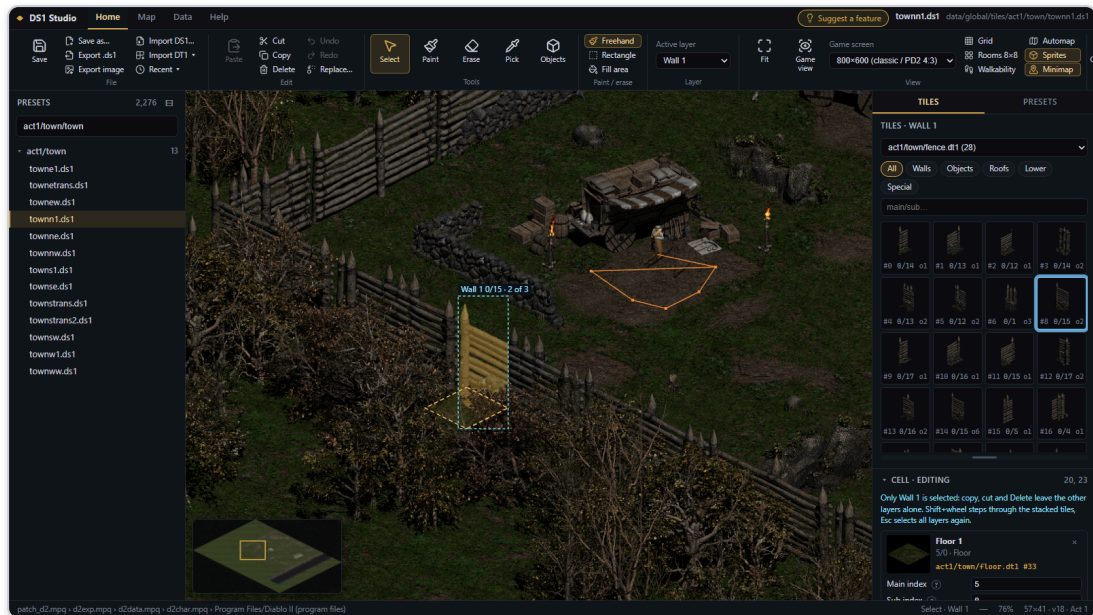
Where map portals arrive A player who comes in without a warp (through a map item's portal) is put, by the game, at a **waypoint** if the level has one, else in a room with a warp tile, else in the room at the **centre of the level**, on the nearest free ground there; if that room has no floor the game stops. The Map entry tile doesn't change this. So place a waypoint where players should arrive: the map can then stay any size, with its tiles anywhere. The [Compatibility check](#) asks for one (**Place a waypoint** puts it on the cursor), and **Add to game** warns at the top when the centre is empty, with **Place a waypoint** and, as an alternative, **Crop the map to what's painted**. For arrivals the waypoint only has to be there; whether players can also travel with it depends on the level's waypoint slot (Levels.txt `Waypoint ; 255 = none`). New maps start with the game's special-tile library, `act1/barracks/warp.dt1`, as maps usually load.

Roof hiding

When a player walks into a building, the game fades its roof. This is set by a **hide area**: two special tiles (main index 8–29) at opposite corners of the inside, whose sub index is the *main index* of the tiles that fade. Every wall-layer tile with that number in the area, plus one cell around it, fades while the player stands inside. The trigger is the rectangle, grown or shrunk toward the south and east by LvlPrest's `PopPad` (in sub-tiles; the game's houses use `-4`). The game ignores hide areas unless LvlPrest's `Pops` counts them.

- **View** → **Roof hiding** (`H`) fills the area the player must stand in, outlines the area whose tiles can fade, and marks those tiles.
- **View** → **Show** → **As if inside** (`Shift` + `H`) hides those tiles, as the game shows the map while a player is inside.
- **Map** → **Roof hiding...** (or **Change roof hiding...** in the Roof hiding panel): select the inside of a building (the floor players walk on), open it, tick what should fade (roofs are ticked for you), and click **Add hide area**. DS1 Studio places the corner markers and sets `Pops / PopPad`. The dialog also lists and removes existing areas.
- **Map** → **Wall hiding...** is the same with the **walls** ticked instead of the roof: the walls in front of a room (usually its south and east walls) fade while a player is inside, so they don't block the view. It uses the same hide areas and the same limits (at most 4 kinds of tile per area).
- The [Compatibility check](#) reports hide areas the game would ignore (`Pops` 0 or too small) and markers it would merge into one rectangle, such as the same number used for two buildings.

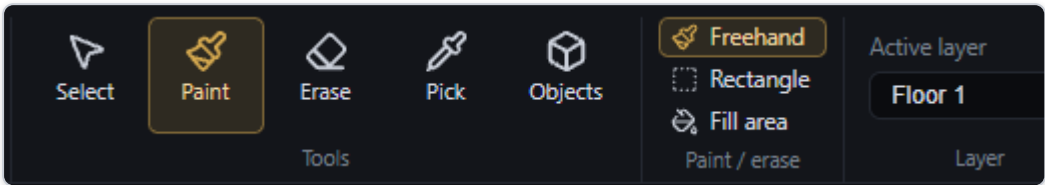
2.8 Overlapping (stacked) tiles



Shift + wheel picks one tile out of several that overlap (the chosen one is outlined).

Trees over trees, a wall over a floor: when several tiles overlap under the mouse, hold **Shift** and turn the mouse wheel to step through them one at a time. Only that tile's layer is selected, so Copy, Cut and Delete leave the others alone; **Esc** selects all layers again. Clicking a tile also shows it in its DT1 in the [Tiles panel](#).

3.1 The tools



Tools, the Paint / erase modes and the active layer.

Tool	Key	What it does
Select	V	Click a cell to inspect and edit it in the Cell panel ; drag to select an area; Shift +click or Shift +drag adds to it (3.4).
Paint	B	Paints the tile chosen in the Tiles panel on the active layer.
Erase	E	Clears cells on the active layer.
Pick	I	Click a tile on the map to paint with it.
Objects	O	Select, move, add and delete objects and NPCs (chapter 4). Shift + O switches between tile and object editing.

The **Active layer** list chooses which layer Paint, Erase, Fill and Clear work on. Floors take floor tiles; wall layers take walls, roofs, lower walls, trees, objects and special tiles.

3.2 The Tiles panel



Pinned and recent tiles on top, a brush plus two mixed tiles (dashed).

- The Tiles panel shows every tile the map's libraries provide for the active layer.
- **Click** a tile to paint with it (switches to Paint).
 - **Ctrl+click** more tiles to paint a **random mix**: each painted cell gets one of them. *clear mix* in the header goes back to one tile.
 - **Right-click** a tile to **pin** it (★) to the strip at the top; **Recent** lists what you used last. Both are remembered per tile set (level type).
 - The drop-down shows one DT1 at a time; on wall layers the chips filter walls, objects, roofs, lower walls and special tiles; the box finds a tile by *main/sub* number.
 - **Ctrl**+wheel over the panel zooms the thumbnails.
 - **Rest the pointer** on a tile for an enlarged view of it beside the panel (crisp pixels, up to 3×) with its size, variants, source DT1 and what clicking does. Thumbnails and this view are shown a little brighter than the game's dark palettes; the map and the files are unchanged.

3.3 Painting: freehand, rectangle, fill

Paint and Erase work in three ways, chosen in the **Paint / erase** group:

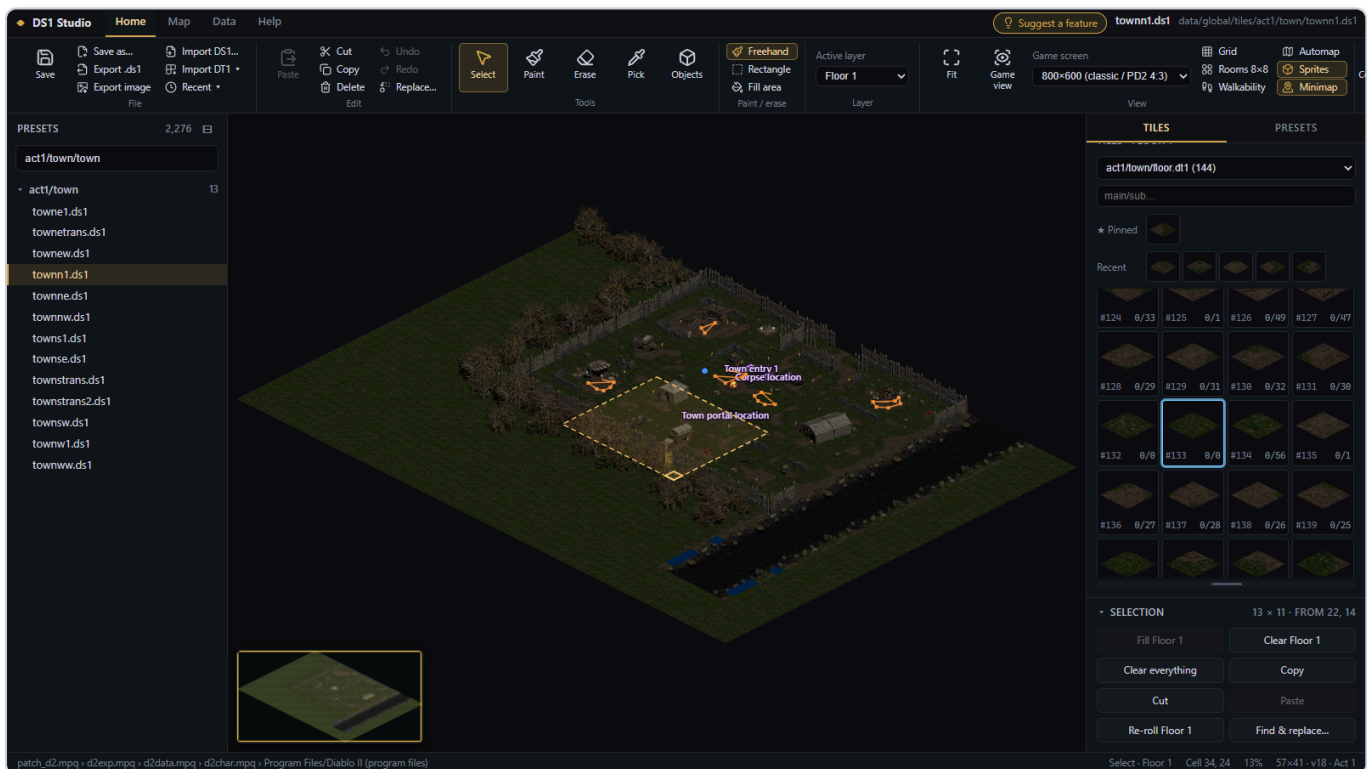
Mode	Key	How
Freehand	B	Paints the cells the mouse passes over.
Rectangle	U	Drag a rectangle; it's all painted (or erased) when you let go.
Fill area	L	Click a cell: the connected area of the same tile is filled. If you click inside a selection, the fill stays inside it.



A rectangle of floor painted with a random mix of two tiles.

Tip Every stroke, rectangle or fill is one step to undo (3.6).

3.4 Selecting, copying and moving



A selection with the Selection panel on the right.

With **Select** (**V**), drag over the map to select cells, or press **Ctrl** + **A** for the whole map. Hold **Shift** to add to the selection: **Shift** + click adds one cell, **Shift** + drag adds a rectangle, so you can select any shape (an L-shaped room, a winding path). Everything below then works on just the selected cells. The **Selection** panel then offers:

- **Fill** the active layer with the brush (or random mix); **Clear** the active layer; **Clear everything** (all layers and the objects standing there, **Shift** + **Delete**).
- **Copy** (**Ctrl** + **C**) and **Cut** (**Ctrl** + **X**) take what you see: layers switched off in Layers, and roofs hidden by **As if inside**, stay where they are, so copying the inside of a building takes its floor and walls, not its roof. Straight after, the copied block follows the mouse to show where it would go: click to paste (**Ctrl** + **V** starts pasting again later). A message at the bottom says what was copied, and the **Copied** panel on the right shows it, with **Paste again** and **Save as preset...** **Esc** stops pasting, closes the panel and deselects. Objects and NPCs in the selection come along, so **Cut then Paste moves an area with everything in it** — even the whole map.
- **Re-roll** and **Find & replace...** (3.5).

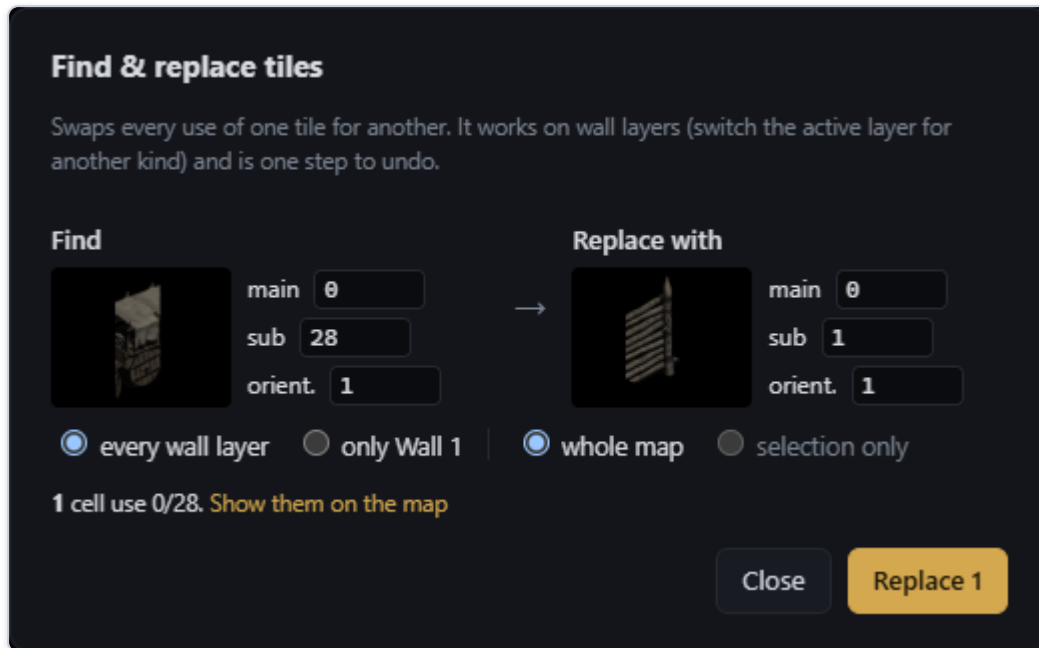
One layer of a selection. After selecting an area, **Shift** + scroll narrows it to one layer at a time (Floor 1, Wall 1, ..., then all layers again): only that layer's tiles stay highlighted, and Copy, Cut and Delete act on just them. For example, select 10×10 cells, scroll to Wall 1, and **Delete** removes only the Wall 1 tiles.

Which layer a tile goes on. With a tile ready to paint, **Shift** + scroll moves it to the next or previous layer of its kind; the preview under the cursor shows it in front of or behind the tile already at that spot (a wall on a lower layer is drawn behind the higher layers' tiles there).

What a paste replaces. While a paste or preset follows the mouse, the cells whose existing tiles it would replace are shaded red, and a note at the bottom counts them, before you click. Hold **Alt** when clicking to stack instead of replacing.

Empty cells paste as transparent. Hold **Alt** when you click to paste to **stack** the paste onto what's there (into the next free layer, adding layers as needed) instead of replacing it. Pasting into another map that doesn't load the copied tiles' DT1s offers to add them; if one of them has the same file name as a different DT1 the map already loads, it asks for a new name first (two libraries with one name get mixed up, placing the wrong tiles) and adds a renamed copy. **Esc** first drops what's on the cursor (a paste, an object, the paint tile), then deselects.

3.5 Find & replace, re-roll



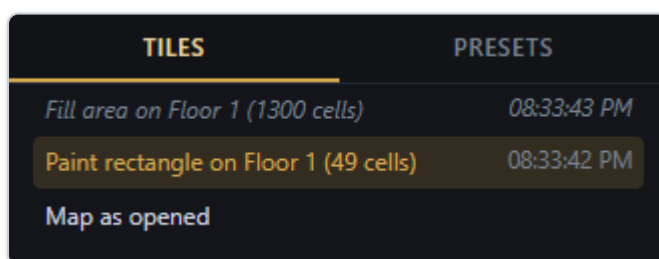
Find & replace tiles.

Find & replace (**Ctrl** + **H**, or **Replace...** on the ribbon) swaps every use of one tile for another, on every layer of the active kind or only the active layer, in the whole map or the selection. It starts with the selected cell's tile and your brush; **Show them on the map** marks every match. One step to undo.

Fill a selection with one tile: select an area with the Select tool, then click a tile in the Tiles panel: every selected cell gets it, on the active layer of its kind (floors on a floor layer, walls on a wall layer, shadows on the shadow layer), as one undo step. Deselect (**Esc**) first to paint with the tile instead.

Re-roll (Selection panel) mixes up the active layer inside the selection: each tile becomes a random tile of the same group (main index) already used there, so repeats are less visible. It never brings in tiles that weren't there.

3.6 Undo and the History panel



The History panel: the current step is highlighted, undone steps are in italics.

Ctrl + **Z** undoes and **Ctrl** + **Y** redoes, without limit. The **History** panel lists every step by name ("Paint rectangle on Floor 1 (36 cells)", "Paste 57×41 + 32 objects"...); click one to go back — or forward — to just after it. *Map as opened* undoes everything. Undoing back to the saved state counts as saved again.

3.7 The Cell panel



The Cell panel for a selected cell.

Click a single cell with Select to edit everything in it. Each layer that has a tile is a card with the tile's picture and the DT1 it comes from (click the file name to show it in the Tiles panel). Every field has a ? that explains it:

- **Kind (orientation)** on wall layers: left/right wall, corner, door, pillar/tree, roof, lower wall, special tile...
- **Main index** and **Sub index**: the tile's number.
- **Present (prop1)**: 00 means empty.
- **Hidden in game**: the game won't draw the tile.
- **Raw bytes**: all four property bytes, to set every bit by hand.
- **Tag value** on maps with a tag layer.

Empty layers are one line with **Add** (puts your brush tile there). × clears a layer. Warp tiles also show where they lead ([chapter 6](#)).

3.8 Walkability: sub-tile flags

Painting walkability on the map (this map only). Switch to **View** → **Walkability** (W): the side panels make way for the **Walkability** panel and the map shows every sub-tile (5×5 per cell) that blocks walking (amber) or jumping and teleporting too (red). Choose **Blocked** or **Walkable**, what it blocks or allows (walking, jumping & teleport, sight & light, players only) and a brush (1×1, 3×3, 5×5 sub-tiles or a whole cell), then click or drag

over the map. **Shift**+drag paints a rectangle and holding **Ctrl** does the opposite. Each stroke is one undo step; **Done** brings the other panels back.

It changes nothing for other maps. The game adds up the sub-tile flags of every tile in a cell, hidden tiles included, so blocking puts an invisible floor tile holding just those flags into a free floor layer of the cell (like Tal Rasha's tomb's own blockers), and making a sub-tile walkable that a tile blocks gives that cell its own copy of the tile without the flag (same picture and variants; a north-corner wall's other half comes along). Both go into a small tile library made for the map, `<map>_walk.dt1` next to it, which is added to the map's tile libraries and level type. A cell whose two floor layers both hold real tiles has no room for a blocker; the panel says which cells it skipped.

Changing a tile everywhere. To change the flags of a tile itself, for every cell and map using it, use the Cell panel:



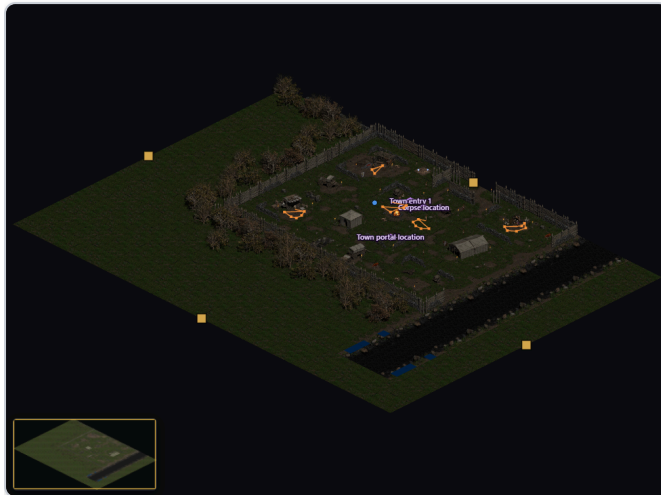
Painting sub-tile flags in the Cell panel.

Every tile has 25 **sub-tiles** (a 5×5 grid over its floor diamond) whose flags decide where units can walk, see, shoot and teleport. Under each card, **Sub-tile flags (walkability)** shows them on the tile:

- 1 Pick a flag (Block walk, Block light & sight, Block jump/teleport, Block player walk, Block missiles...). Hover a flag for what it does.
- 2 Click or drag over sub-tiles to set it; start on one that has it to clear instead.
- 3 Check the result on the map with **Walkability** (W).
- 4 **Save DT1s to mod** in the bar at the top of the panel, or **Discard**.

Important The flags belong to the tile in its DT1: every cell and map using that tile changes. *Change all variants* (on by default) keeps the random variants of the tile alike.

3.9 Resizing and new maps



Map → Resize: drag the handles on the edges.

New map

Save path

data/global/tiles/expansion/Custom/newmap.ds1

Size

150 × 150 tiles

Act

Act 5 (new levels added to the game are Act 5)

Layers

1 floor 2 wall ☐ tag layer + groups

The map starts with no tile libraries: the DT1 library opens next, to choose them from any act. It's shown in the **Act 0** colours (the ones that look the same in every act; magenta marks colours that change between acts). **Game** → **Add to game** later makes the game load it, with a level type of its own when it needs one.

Cancel

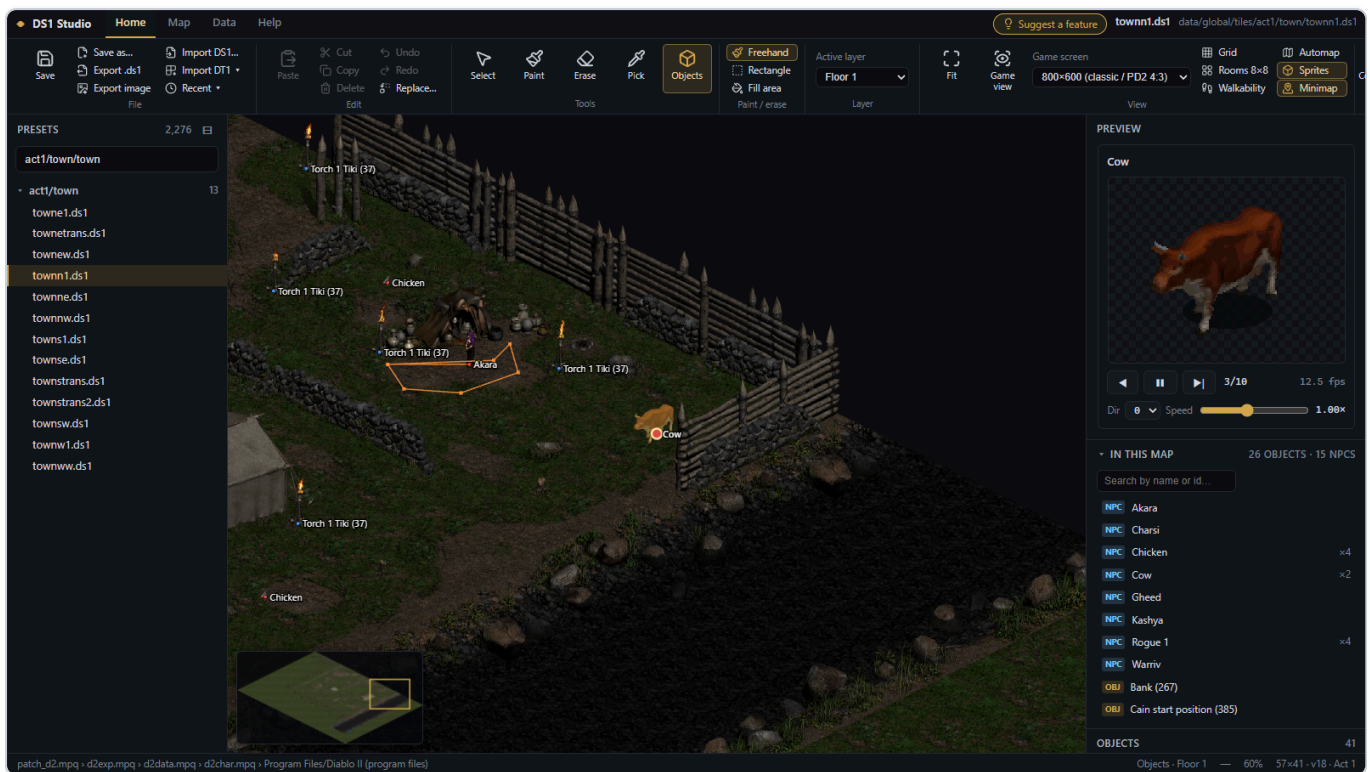
Create

Map → New map.

Map → **Resize** shows handles on the four edges: drag them to add or remove cells (objects move with the cells). **Resize...** does it by numbers. When the map is a whole level of its own, saving it after a resize also sets that level's size in Levels.txt (SizeX/SizeY, all difficulties) to match: the game stops with an error building a level whose map isn't its size. **Map** → **New map** creates an empty map, 150×150 by default, with no tile libraries: the DT1 library opens straight after, to choose them from any act. A new map is shown in the **Act 0** colours (only the colours that look the same in every act; magenta marks those that change between acts), so tiles from different acts look as they will in game; **View** → **Colours** switches palette, and a map remembers the one you pick. Its act is Act 5 by default, the act the game gives every level added with Add to game. It's saved where *Save path* says. **Level type** chooses where its tiles come from (hover the ? next to each): **(New Lvtype)** starts empty as above and gets a level type of its own when it's added to the game, listing just the tile libraries it uses; **(Choose Existing)** starts with an existing level type's tile libraries (from LvTypes.txt) in that act's colours, and Add to game then starts from a level of that type. Use **Game** → **Add to game** to make the game load it; that also gives the map a level type of its own, holding just its tile libraries, when it needs one.

4 Objects and NPCs

[↑ Contents](#)



Object mode (**O**): the preview, "In this map" and the gallery on the right.

4.1 Placing from the gallery

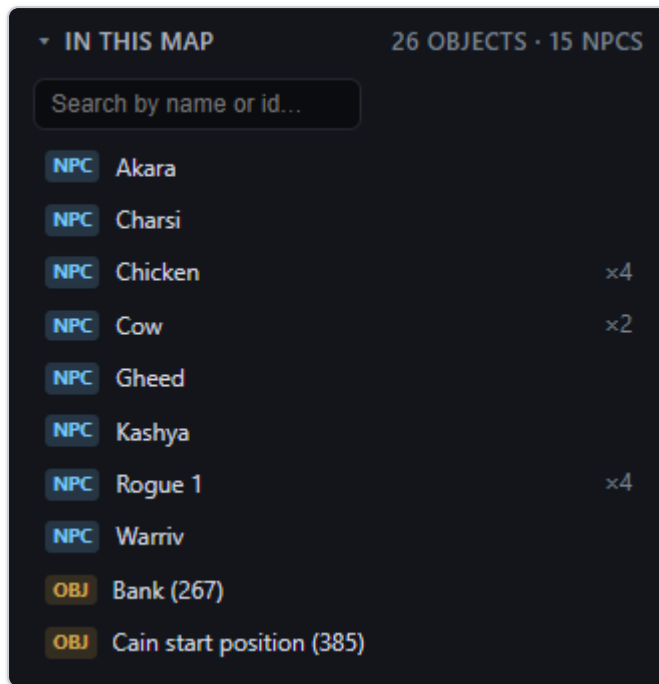
- 1 Switch to **Objects** (**O**), or **Shift** + **O** from a tile tool.
- 2 In **Objects & NPCs**, filter by *NPCs* or *Objects* or type a name; every one is shown with its sprite.
- 3 Click one, then click the map to place it. Keep clicking to place more; **Esc** stops.

The list is the act's placeable objects and NPCs, worked out from the game's own tables (and your mod's).

4.2 Moving and editing objects

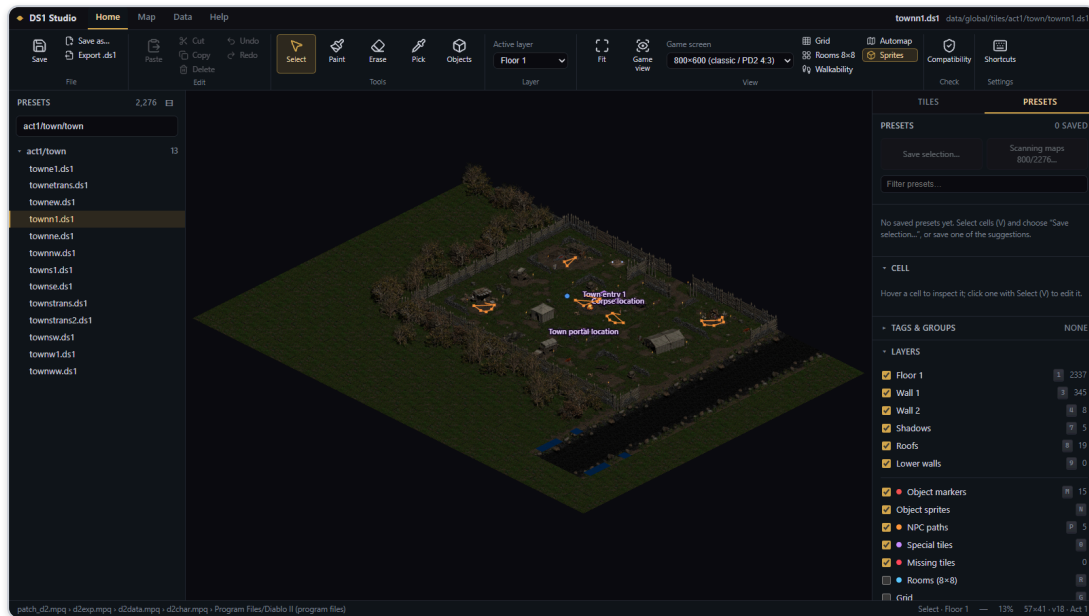
In object mode, drag an object to move it; **Delete** removes the selected one. **Double-click** an object to select every object of the same kind on the map (all 19 torches, say): **Delete**, **Ctrl** + **C** and **Ctrl** + **X** then act on all of them. Switching to another view deselects objects. **Ctrl** + **C** copies the selected object onto the cursor, shown in **green**; **Ctrl** + **X** cuts it, taking it off the map and showing it in **red**. Click the map to put it down (an NPC's path moves with it); **Ctrl** + **V** brings it back for another copy, and **Esc** drops it from the cursor. The name a player sees when pointing at an object comes from the game's string tables; a few objects (the guild objects Blizzard cut, such as Guild Vault and Steeg Stone) have none and show an empty box, which the [Compatibility check](#) finds and fixes by adding the names to your mod's `patchstring.tbl`. The **Preview** panel animates the selected object in every direction. The **Object** panel edits its type, id, position, flags and its NPC path (the points an NPC walks between).

4.3 “In this map”: find any object



Every object and NPC placed, with counts.

In this map lists everything placed, grouped by name with a count (Chicken x4...). Type to search by name or id. Click a name to select it and centre the view on it; click again for the next one of that kind.



The Presets panel with suggested presets.

A **preset** is a reusable piece of map — a building, a wall run, a group of trees.

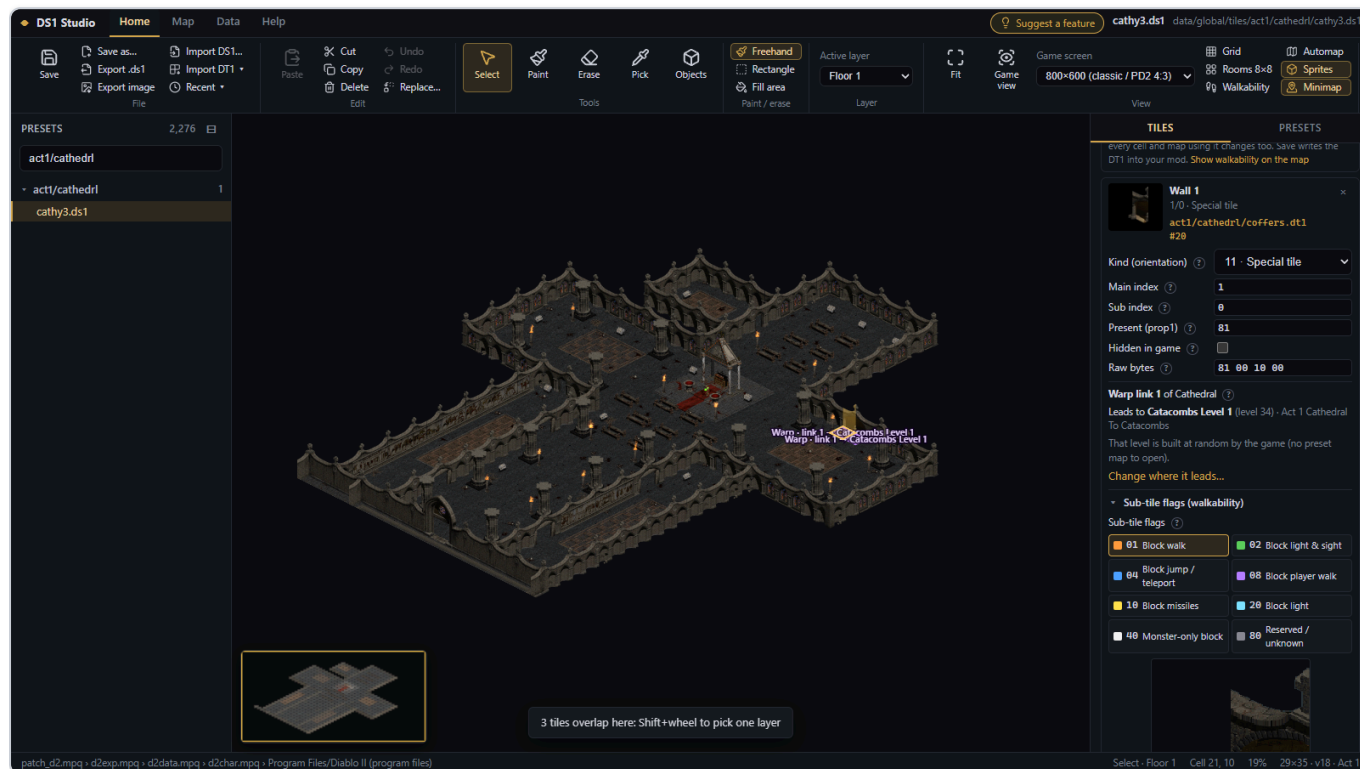
- **Save selection** (Map → Presets, the Presets panel, or **Save as preset...** in the Copied panel after a copy) stores the selected area as a preset in your mod folder, so it travels with the mod. The **Save as preset** window asks for a name and a category, shows a picture of the preset, and lets you choose what it keeps: **Floors, Walls, Roofs, Shadows, Markers** (special tiles) and **Objects & NPCs**, each with how many there are. It starts from what you see: with **As if inside** on, Roofs starts unticked, so the inside of a building saves without its roof; with it off everything is ticked, and one click on Roofs leaves them out. Layers switched off in Layers start unticked too.
- **Suggest** finds structures that recur across the maps sharing this map's tiles and offers them as presets.
- Click a preset, then click the map to place it (like a **paste**; **Alt** stacks it onto existing tiles). Red cells show the tiles it would replace.
- **Right-click** a saved preset to place, rename, move it to another category, duplicate or delete it (a deleted preset's file is kept aside as a backup). Right-click a suggestion to place it or save it to the mod.
- The **Preset builder** starts with this map's tile libraries; add any other DT1 with **Add a DT1**. Tiles from a library that isn't in the Act 0 colours yet (an added one, or one of the map's) are copied into the preset converted to Act 0, so a preset looks right in any act.

Open the panel with the **Presets** tab next to Tiles, or **Map → Presets**.

6 Warps: connecting maps

[↑ Contents](#)

In Diablo II each map preset is its own **level**, and levels are joined by **warps**. A warp special tile with main index N (0–7) uses its level's **link N**: `Levels.txt` column `VisN` says which level it leads to, and `WarpN` which kind of warp it is (a `LvlWarp.txt` row: stairs, cave entrance...). DS1 Studio reads and edits these for you.



A warp in the Cathedral: the label and the Cell panel show where it leads.

Seeing where a warp leads

- On the map, warp markers read e.g. *Warp · link 1 → Catacombs Level 1*, or *(not set)* when the link is empty.
- Select the warp cell: the Cell panel shows the level it leads to, the kind of warp, and **Open** buttons for that level's maps. Levels built at random by the game have no map to open.

Changing where it leads

Warp link 1 of Cathedral

Warp tiles with main index 1 in this level's maps use this link. Choose the level it leads to and the kind of warp (LvlWarp.txt decides the look and where the player appears). Saved into Levels.txt in your mod.

Leads to

- 26 - Monastery Gate
- 27 - Outer Cloister
- 28 - Barracks
- 29 - Jail Level 1
- 30 - Jail Level 2
- 31 - Jail Level 3
- 32 - Inner Cloister
- 33 - Cathedral
- 34 - Catacombs Level 1

Its maps: none (built at random)

Kind of warp

15 - Act 1 Cathedral To Catacombs

☐ also add the way back (the target level gets a link to Cathedral)

• Cathedral: link 1 → Catacombs Level 1 (level 34), LvlWarp 15

If your mod ships compiled .bin tables, rebuild them after applying (start the game once with -direct -txt).

Remove link

Cancel

Apply

Choosing the level a link leads to.

- 1 Select the warp tile and click **Change where it leads...** (or **Connect it to a level...**).
- 2 Search and pick the level it should lead to, and the kind of warp.
- 3 Tick **also add the way back** to give the target level a link back here (its map needs a warp tile with that link's number).
- 4 **Apply** writes `Levels.txt` in your mod. **Remove link** clears it.

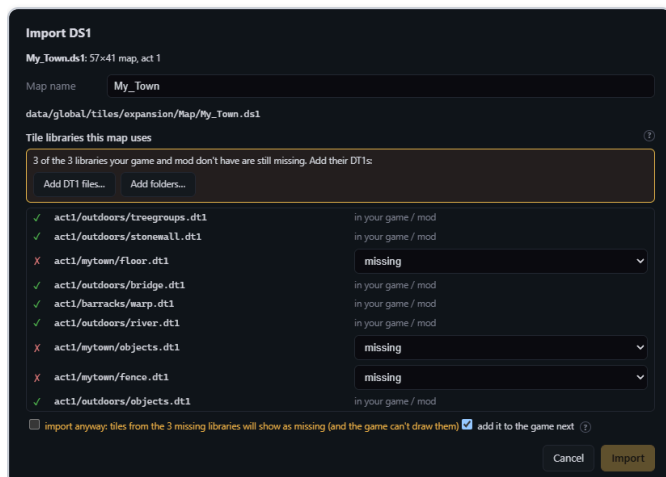
Note A map must be tied to a level before its warps lead anywhere: use [Add to game](#) first. If your mod ships compiled `.bin` tables, rebuild them afterwards ([see 12.3](#)).

7 Importing DS1 and DT1 files

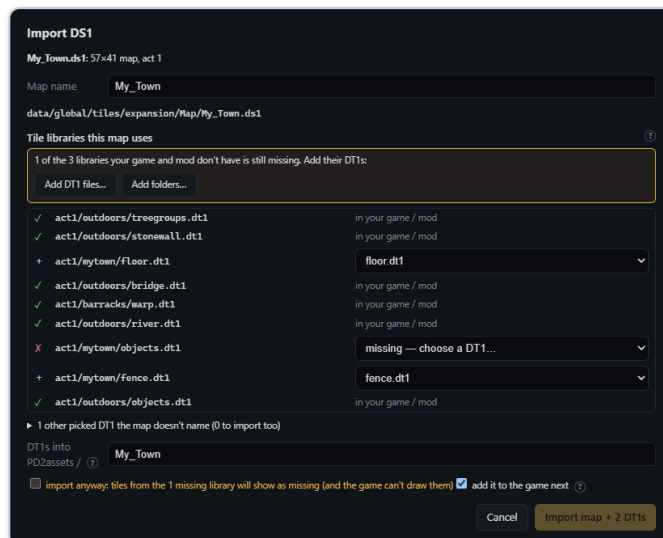
Bring maps and tile libraries from anywhere on your computer into your mod, with the game's tables updated so they load without crashing. Both are in **Home** → **File** → **Import** ▾.

7.1 Import DS1 (with its tile libraries)

- 1 **Import** ▾ → **Map...** and pick the `.ds1` file. It's checked first: a file the game couldn't read is refused. (A map package `.zip` from [Export map](#) can be picked here too: it opens the package import, tables included.)
- 2 A `.ds1` on its own brings none of its rows from the game's tables (Levels, LvlPrest, LvlTypes, CubeMain, AutoMap), so a warning says your own are used as they are: if they don't match the map and its tile libraries, the game can crash when it loads or while walking in it. Add it to the game afterwards and run the Compatibility check.
- 3 Give the map a name (letters, digits, `-` and `_`). It will be saved as `expansion/Map/<name>.ds1`.
- 4 Check **Tile libraries this map uses**: every DT1 the map names is ✓ in your game/mod, + provided by a DT1 you picked, or ✗ missing.
- 5 For missing ones, click **Add DT1 files...** or **Add folders...** (subfolders included). They're matched by file name, using the folders to tell same-named files apart; you can also choose a file for any library from its list.
- 6 If the map names the same tiles twice in different folders (an original and a copy), a warning explains that the map would show a random mix of them; **keep one copy of each** (on by default) leaves out the first-listed copies. A picked DT1 with the same name as a library you already have can be chosen on that library's line to replace it.
- 7 Import stays unavailable while any library is missing — find those DT1s, or tick **import anyway** (their tiles will show as missing and the game can't draw them).
- 8 **Import**. The provided DT1s go into `PD2assets/<folder>/` (keeping the paths the map names) and the map is pointed at them. With **add it to the game next** ticked, [Add to game](#) opens, pre-filled with a level that uses the same tiles — click **Apply**.



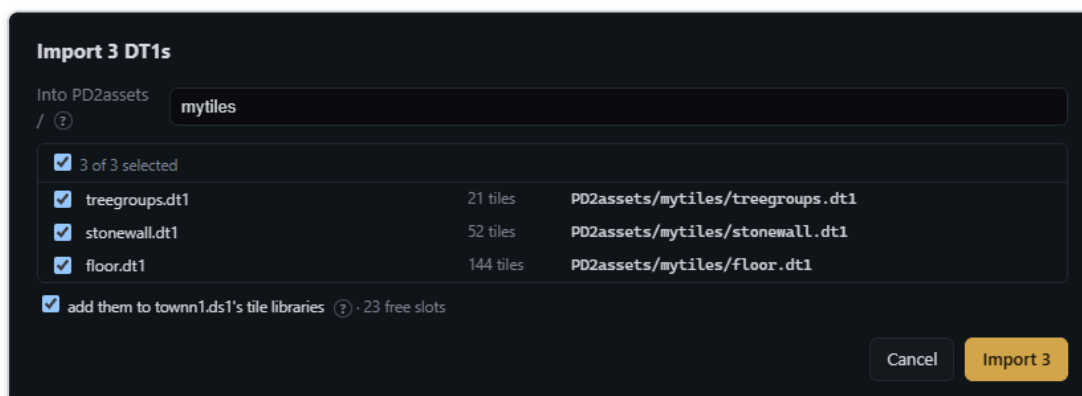
Three libraries the map needs are missing.



After adding DT1 files: two provided (+), one still missing (X), one extra file listed separately.

7.2 The DT1 library: import, browse, build

Everything to do with tile libraries happens in one window: **Home** → **File** → **Import** ▾ → **DT1s...** or **Map** → **DT1 library....** It lists every DT1 the game and your mod have; you can bring in your own, look through any of them, add whole libraries to the map, or build a custom DT1 from single tiles.



Three DT1s ready to import into PD2assets/mytiles.

Bringing in DT1s from your computer:

- 1 In the library window, + **Add DT1 files...** (one or several) or + **Add a folder...** (every `.dt1` inside, subfolders included).
- 2 Choose the folder under `PD2assets/` they go into. *Keep the folder structure* keeps the picked folders' subfolders.
- 3 Check the list: each file's tile count, unreadable files (left out), files that will be replaced (the old one is kept as `.bak`). Untick any you don't want.
- 4 With a map open, **add them to this map's tile libraries** puts each in a free `File` slot of the map's level type and in its `Dt1Mask`, so the game loads them. The free slots are shown; a level type has 32.
- 5 The library then shows the first one you brought in (under `PD2assets/` in its tree).

From the game library

The window lists every tile library the game and your mods already have, by folder, with a search box (the map's own libraries first). Click one to see all its tiles, filtered by kind; rest the pointer on a tile to see it enlarged, click it for its details. The tiles are shown in the library's own act colours; **Colours** switches to the map's act, or to Act 0, which shows in magenta the colours that change between acts. **Choose this library** for each one you want, then **Add to the map. Act 0 is the standard:** with *Convert them to Act 0 colours* ticked (the default), each library that uses colours changing between acts is converted first, so its tiles look right in any act; a library from the game's archives gets its converted copy in your mod, which every map using it shares. The map's level type and `Dt1Mask` are updated as in [Tile libraries](#). DT1 files you import (+ **Add DT1 files...**) are converted as they come in, and custom DT1s are built in Act 0 colours unless you untick it.

Building a custom DT1 from single tiles

Switch the window to **Build a custom DT1** to put together a new tile library from single tiles of any others.

- 1 Open a library and click tiles to tick them (**Shift+click** ticks a range, **Tick all shown** takes everything the filter shows). Go on in other libraries: the tree marks the ones you picked from, and **Picked tiles** collects them all (click one to go back to its library, × to drop it).
- 2 Name it. Keep the name short, 12 characters or fewer is best, using only letters, digits and `_` : spaces, dots, dashes and accents can break the game's tables and archives. The whole path has to fit the game's 41-character limit for tile paths, so the window says how long it may be. It goes into `PD2assets/custom/` unless you change the folder.
- 3 Check the summary, then **Create DT1 and add it to the map.**

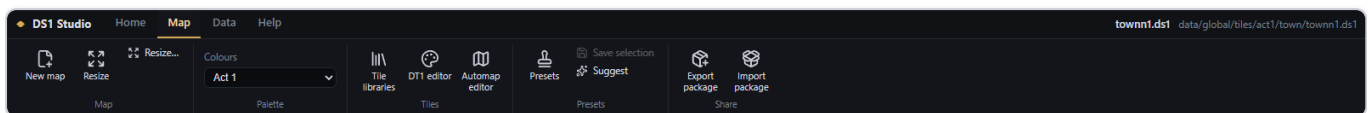
What the new library gets:

- **Each tile exactly as it was:** pixels, walkability (sub-tile flags), sound, roof height and rarity are copied byte for byte.
- **Whole pieces:** a north corner wall brings its other half, and an animated tile all its frames. Special tiles (markers) can't be picked.
- **Numbers that don't clash:** the game finds a cell's tile by its orientation, main and sub index among every library the level loads. A tile whose number the level already uses, or another picked tile has, would be mixed with it at random, so it gets the first free number instead; the summary lists these. Tiles you pick that share a number in their own library stay together as variants.
- **The tables:** the library goes into a free `File` slot of the map's level type and its `Dt1Mask` (a level type of its own if other levels share it, as in [Tile libraries](#)), and each tile's `AutoMap.txt` pieces are copied from the level it came from under its new number. Originals are kept as `.bak`.

Colours A DT1 stores palette numbers, not colours: in game its tiles take the colours of the level's act. Tiles from another act's library can look different there; **Colours → this map's act** shows them as the game will. Tick **Make it act-safe (Act 0 colours)** to fix that: every colour of the picked tiles is snapped to the nearest one that looks the same in every act (Gimli's Act 0 palette; nearest by plain RGB distance, as the community's Act 0 tile sets were made), judged by how it looks in the act its library was drawn for (its folder's: ACT1-ACT4, expansion = Act 5, Guild = Act 1; else the act its art fits best, worked out from the tiles; only failing that this map's). The window says how many picked tiles use act-specific colours, and the picked tiles show how they will look. Resting the pointer on a picked tile shows it enlarged.

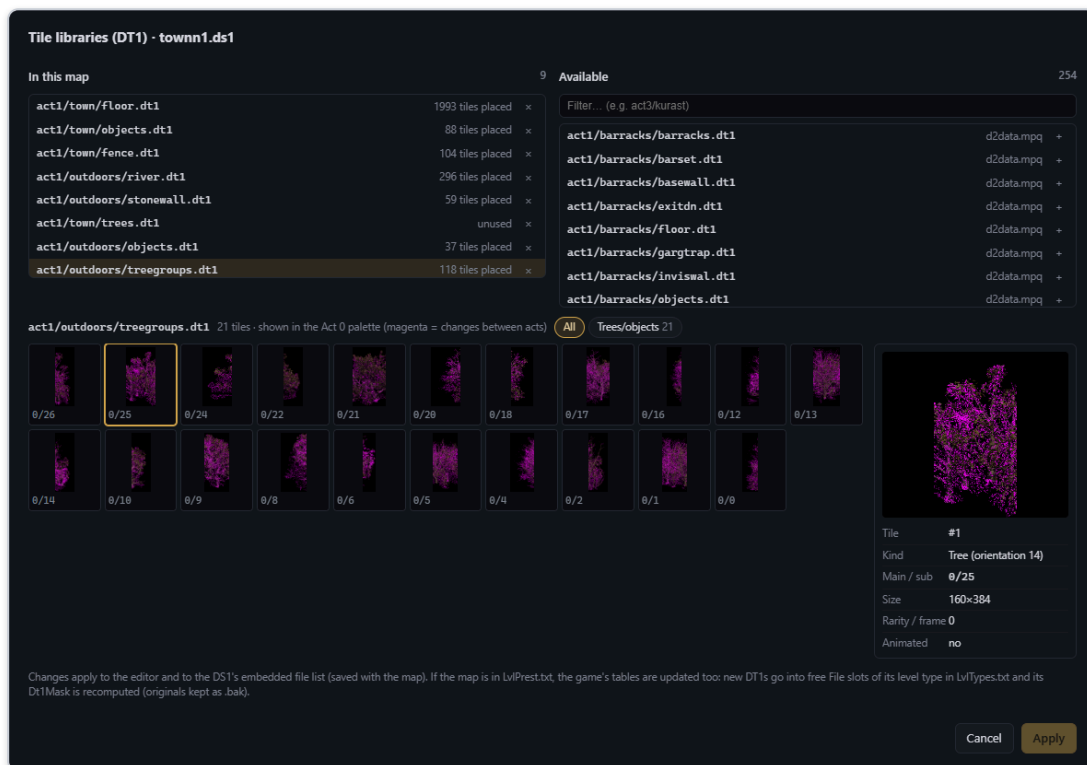
8 Tile libraries and the DT1 editor

[↑ Contents](#)



The Map ribbon.

8.1 Tile libraries



Map → Tile libraries (an earlier version of the window).

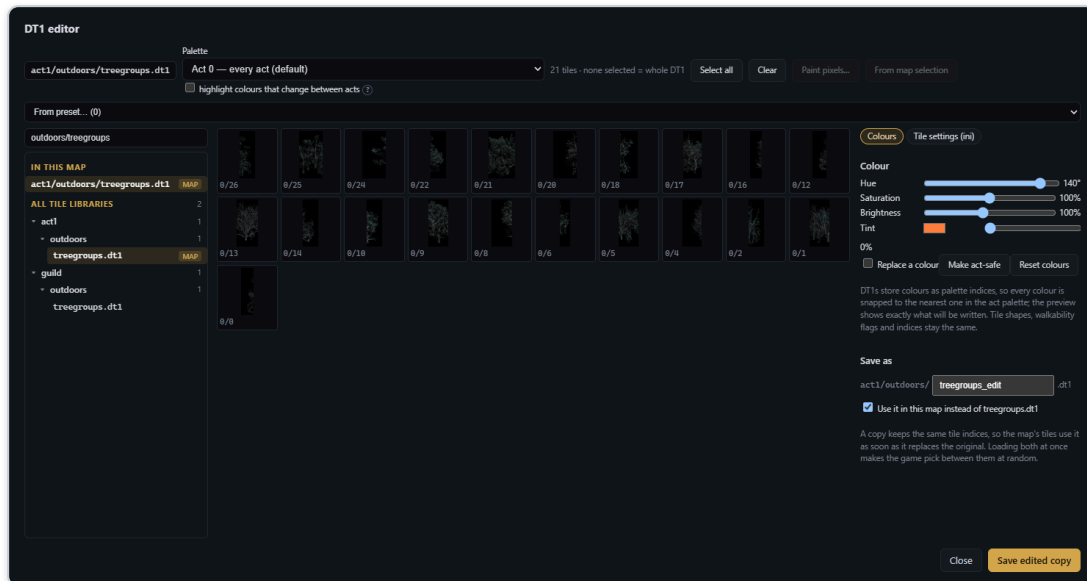
Map → Tile libraries reviews the DT1s the map is attached to. The list says, for each, how many of its tiles the map places (*drawn from a later library* when another library with the same tile numbers is the one the game draws) and whether it is the game's or your mod's. Click one to see its tiles: the ones the map uses are outlined in green with how many cells use them, and **Used in this map** shows only those. For a tile you click:

- **Show on map** marks every cell that uses it.
- **Remove this tile from the DT1...** rewrites the DT1 without it (the other tiles stay exactly as they were; the original is kept as `.bak`). You're told first if the map uses it, or if other level types load the DT1 too.

Rename... renames a DT1 in its folder (letters, digits and `_`): the file is written under the new name, every level type that loads it (`LvlTypes.txt`) and the map are pointed at it, and the old file is moved aside to `.bak`, never deleted. The `x` next to a library takes it off this map (**Remove from the map**); the file stays. **+ Add libraries...** opens the [DT1 library](#). Removing tiles and renaming work on your mod's own DT1s only: one in the game's archives is shared by every map, so make your own copy with the [DT1 editor](#) first.

DS1 Studio keeps the game in step: new DT1s go into free `File` slots of the level type (`LvlTypes.txt`) and the map's `Dt1Mask` (`LvlPrest.txt`) is recomputed. When two loaded DT1s have the same tile, the one listed later replaces it, as in the game.

8.2 The DT1 editor



Map → DT1 editor.

Map → DT1 editor edits a copy of any DT1: choose it in the tree on the left (every library in the game and mod, by folder), select tiles (**Shift** + click for a range, or *From preset / From map selection*), and edit their colours (8.3), pixels (8.4) or settings (8.5). Clicking a tile opens it in a zoom window you can move, resize and zoom to single pixels. **Save** writes a copy under a new name, or overwrites the original, and can switch your map over to it with the tables updated.

Palette Editing uses the **Act 0** palette: only the 225 colours that look the same in every act, so edited tiles work in any act. Tick *highlight colours that change between acts* to see act-specific pixels in magenta; **Make act-safe** converts them.

Make act-safe: fixing odd red or purple tiles

About 30 of the palette's colours change between acts, so tiles drawn for one act (say Act 1) can show red or purple patches in a map of another act, here and in game. **Map → Make act-safe** lists every DT1 the map uses, how many of its pixels use act-specific colours, and a *now / after* preview. For each DT1, choose the act it was drawn for so the *after* preview looks right, then click **Convert**. Every DT1 that still uses such colours is ticked, since Act 0 is the standard. The DT1s are saved into your mod with `.bak` copies, and every map using them gets the new colours.

Random red cells? If only *some* cells show the odd colours, in a random scatter, the map probably loads two copies of the same tiles: an original and an act-safe copy, often from an import. The game picks among both copies' variants at random. Make act-safe and the [Compatibility check](#) both detect this ("tile libraries loaded twice") and remove the extra copy from the map's libraries, so no DT1 needs converting.

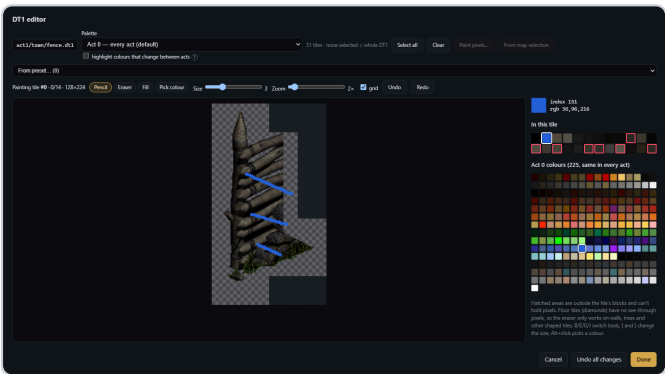
8.3 Recolouring



Changing the hue of a group of tiles.

Hue, saturation, brightness, tint, and **replace a colour** — for the whole file, the selected tiles, a preset's tiles, or the tiles in your map selection. The preview is exact.

8.4 Pixel painting

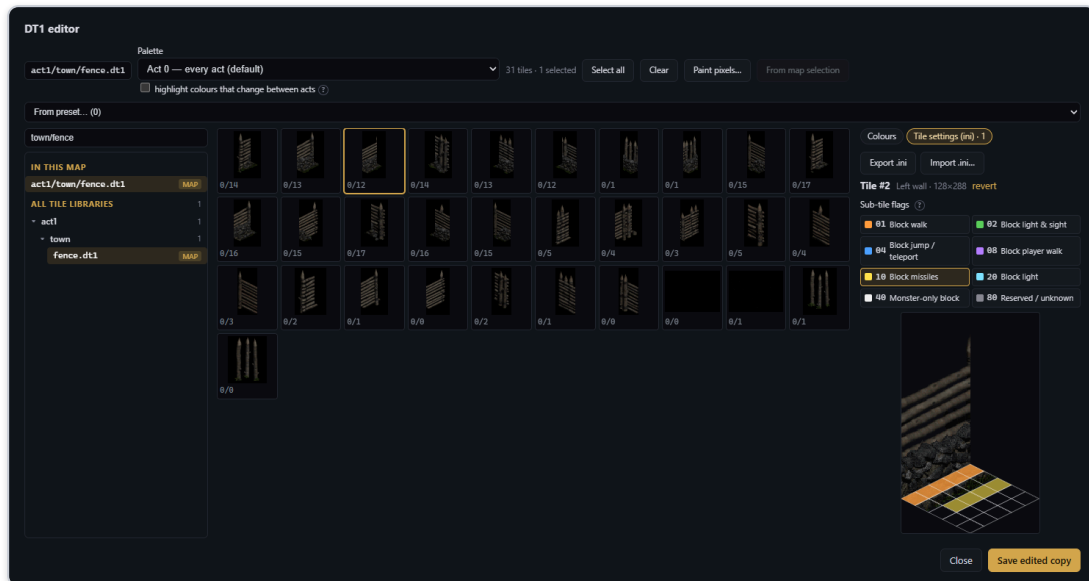


The pixel painter with the Act 0 colours.

Double-click a tile (or **Paint pixels...**) to paint it pixel by pixel. Only the tile's real pixels can be painted.

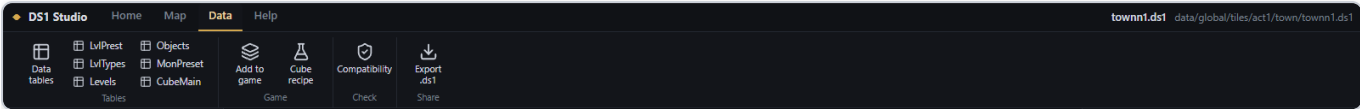
Pencil / eraser / fill / colour picker	B E G I
Brush size	[]
Pick a colour	Alt + click
Undo / redo	Ctrl + Z / Ctrl + Y
Zoom	Ctrl + wheel

8.5 Tile settings (.ini)



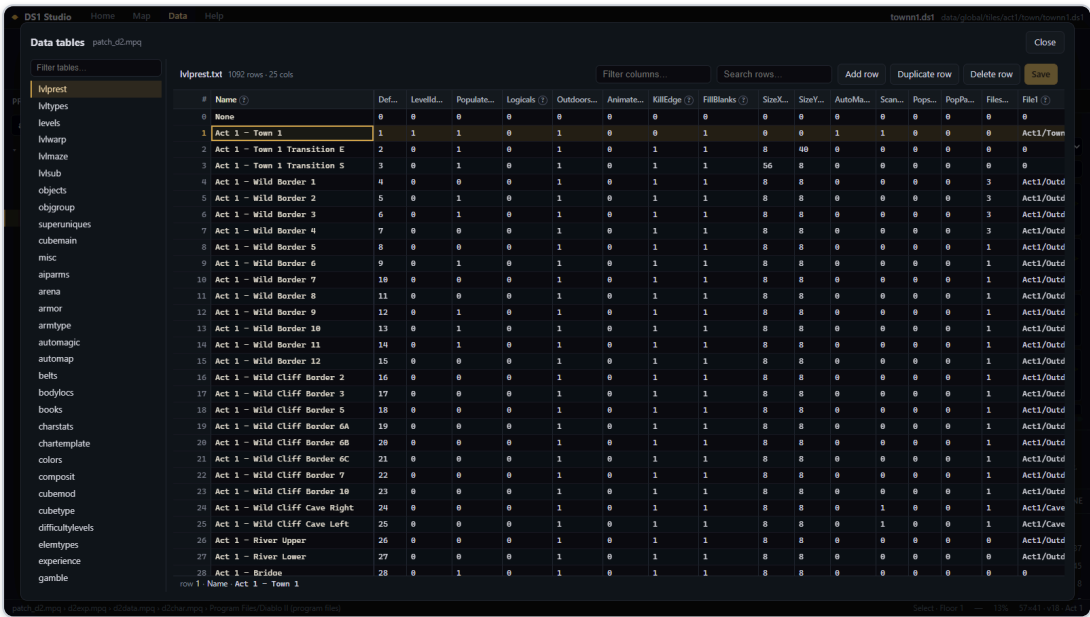
The Tile settings tab of the DT1 editor.

The **Tile settings (.ini)** tab edits everything a DT1 Tools `.ini` holds for each tile — kind, main/sub index, rarity/frame, animated, footstep sound, roof height, direction — for one tile or many at once, each with an explanation on hover. The **sub-tile flags** are painted on the tile's 5×5 grid. **Export .ini** / **Import .ini** exchange settings with DT1 Tools.



The Data ribbon.

9.1 Data tables



Game → Data tables.

Browse and edit any `.txt` table of the game in a spreadsheet view. Column headers have a `?` explaining what the column does. The shortcuts (LvIPrest, LvTypes, Levels, Objects, MonPreset, CubeMain) jump to the open map's row where there is one. Saving writes the table into your mod.

9.2 Add to game

Add map to game

Makes the game load `act1/town/townn1.ds1`: a level for it in `Levels.txt`, the `LvlPrest` row (?) that builds the level from this map, and the tile libraries it uses in its level type's `LvlTypes` slots (?) with the matching `Dt1Mask` (?). Every field is listed below before anything is written.

Level ☒ a new level, with settings copied from ☐ replace the map of an existing preset level

Copy from (?) 1 - Act 1 - Town

Name townn1

Colours (?) Act 1 palette (the act the tiles are from)

New levels are Act 5 levels in the game (it goes by the level number), so they use Act 5's objects, music and town. The palette follows the act the tiles were drawn for, as the game's own Act 5 levels that reuse other acts' tiles do.

The level is in Act 5 but uses the Act 1 palette (Pal 0), because Act 1 - Town's tiles were drawn for that act.

Levels.txt

Level 137 "townn1" (new row) copy of 1 Act 1 - Town, appended as record 137

Name	Act 1 - Town → townn1	
Id	1 → 137	its row number: the game reads Levels.txt by row
Act	0 → 4	levels from 109 on are Act 5 in the game
Quest	0 → (empty)	
Layer	0 → 100	its own automap
OffsetX	-1 → 5900	a free spot in Act 5, away from the other levels and inside the automap's range
OffsetY	-1 → 2050	
Waypoint	0 → 255	no waypoint
LevelName	Rogue Encampment → townn1	
LevelWarp	To The Rogue Encampment → townn1	
EntryFile	A1L1 → A5L1	the loading screen (data/local/ui/eng/expansion/A5L1.dc6, Harrogoth's)

LvlPrest.txt

Preset 1091 "townn1" (new row) record 1091 for level 137

The new level has no connections yet: link it from another level (select a warp tile → Change where it leads) or open it with a cube recipe (Game → Cube recipe). If your mod ships compiled .bin tables, rebuild them after applying (start the game once with `-direct -txt`).

Cancel Apply

Game → Add to game.

Makes the game load the open map. The dialog lists every field it will write, with the old value, the new value and the reason, before you click **Apply**. There are two ways to add a map:

- **A new level**, with settings (monsters, lighting, sound, tile set) copied from a level you choose. A `Levels.txt` row is added as the next record, and its `LvlPrest.txt` row points at the map. **Level type** chooses between *a new level type* (a `LvlTypes.txt` row of its own, listing just the map's tile libraries) and *the one it copies from* (a new one is still made if that type lacks libraries the map uses and other levels share it). A map made with **(New Lvlttype)** in New map starts with the first ticked.
- **Replace the map of an existing preset level**. The level's own `LvlPrest` row is pointed at the map. Only preset levels can do this: levels the game builds at random don't load a single map.

Automap layer A new level keeps the `Layer` of the level it copies (PD2's maps all share 98). The game takes layers up to 100: a level with a higher one crashes the game the moment a player enters it (earlier versions of DS1 Studio counted up past it). The [Compatibility check](#) reports such a level and sets it back.

Either way, the tile libraries the map uses go into free `File` slots of the level type (`LvlTypes.txt`), and the preset's `Dt1Mask` selects them. The rules it follows come from the game's own code and tables:

Rule	Why
New rows are always added at the end: a level's <code>Id</code> , a preset's <code>Def</code> and a level type's <code>Id</code> equal their row number. The <i>Expansion</i> line doesn't count.	The game reads these tables by row position. A row inserted in the middle shifts every level after it.
A level uses the <i>first</i> <code>LvlPrest</code> row with its <code>LevelId</code> .	A second row for the same level is ignored, so an existing level is changed through its own row.
New levels are Act 5 (<code>Act</code> 4).	The game takes the act from the level number: 109 and up is Act 5. That sets the objects, music and town.
<code>Pal</code> = the act the tiles were drawn for (you can change it).	<code>Pal</code> picks the colours. The game's own Act 5 levels that reuse Act 1, 2 and 4 tiles do this.
<code>SizeX</code> / <code>SizeY</code> (all difficulties) = the map size minus one. <code>LvlPrest</code> <code>SizeX</code> / <code>SizeY</code> stay 0.	As in every preset level of the game. The preset then fills the level.
<code>OffsetX</code> / <code>OffsetY</code> at a free spot. New <code>Layer</code> (automap). Quest requirement, waypoint (255), links (<code>Vis</code> 0 / <code>Warp</code> -1) cleared.	Levels of one act share a world and must not overlap. The copied level's quest lock or waypoint would otherwise carry over.
<code>FillBlanks</code> 1, <code>Scan</code> 1, <code>AutoMap</code> 0, <code>Files</code> 1. <code>Populate</code> / <code>Outdoors</code> / <code>KillEdge</code> as the copied level's preset. <code>Pops</code> / <code>PopPad</code> for roof hiding .	<code>Scan</code> finds warps and special tiles. <code>AutoMap</code> 1 would reveal the whole automap on entry, as in towns.
Tile and map paths are at most 41 characters.	The game adds <code>DATA\GLOBAL\TILES\</code> in front, in a 60-byte buffer. The game's own longest path is 40.

The [Compatibility check](#) checks the same rules for any map already in the game, and offers one-click fixes: rows out of order (put back in Id order; no Id changes), a preset pointing at a missing level, the level size, act, palette, an overlapping position, the file count and a copied quest requirement. A new level has no connections yet: link it with [warps](#) or open it with a [cube recipe](#). Set the DS1 header to Act 5 (the check offers it), so the map's object numbers are read as Act 5 objects.

9.3 Cube recipe

Cube recipe for this map

Makes a new map item and a Horadric Cube recipe that creates it. Which map the item opens is decided by your mod's map system (for PD2, its map items) using the item's code.

1 - The map item

From a template Custom item (my own code)

Template item ?

Search by name or code...

OTHER ITEMS

elixir elx

Healing Potion hpo

Mana Potion mpo

Full Healing Potion hpf

Full Mana Potion mpf

Stamina Potion vps

Item name ?

Item code ? made for you (unused)

2 - What goes in the cube ?

Town Portal Book tbk x 1 x

Identify Scroll isc x 1 x

+ Add ingredient

► Advanced: type cube inputs yourself ?

What will be written ?

Misc.txt: New item "townn1 Map" (code to01), copied from "Healing Potion"

CubeMain.txt: Recipe: Town Portal Book + Identify Scroll → townn1 Map

Cancel Create item and recipe

Game → Cube recipe.

Creates a new item and a Horadric Cube recipe that makes it. Pick the template item and the ingredients from lists with icons, or type your own item code (checked against the game's items). DS1 Studio takes care of the rules that otherwise crash or break the game:

- **Placement:** the new item is added at the **end** of `Misc.txt`. Items are numbered by their position, so one inserted in the middle rennumbers every item after it.
- **No duplicate recipes:** a recipe with the same ingredients as an existing one is refused. The cube only runs the first match, so the new one would never work.
- **Never a random drop:** the new item gets `spawnable 0`, and references to the template's code (`normcode ...`) are changed to the new code.
- **Safe templates only:** elixirs aren't offered; a copy of one crashed the game when hovered. Items of a type the base game doesn't have, like PD2's maps, need a tick first. The mod's own code handles those types and only knows the mod's own item codes.

A map item that opens this map

Pick one of your mod's **map items** as the template; they are listed first under *Map items (open a level)*. The new item then opens **the map you have open**, and DS1 Studio makes every change for it:

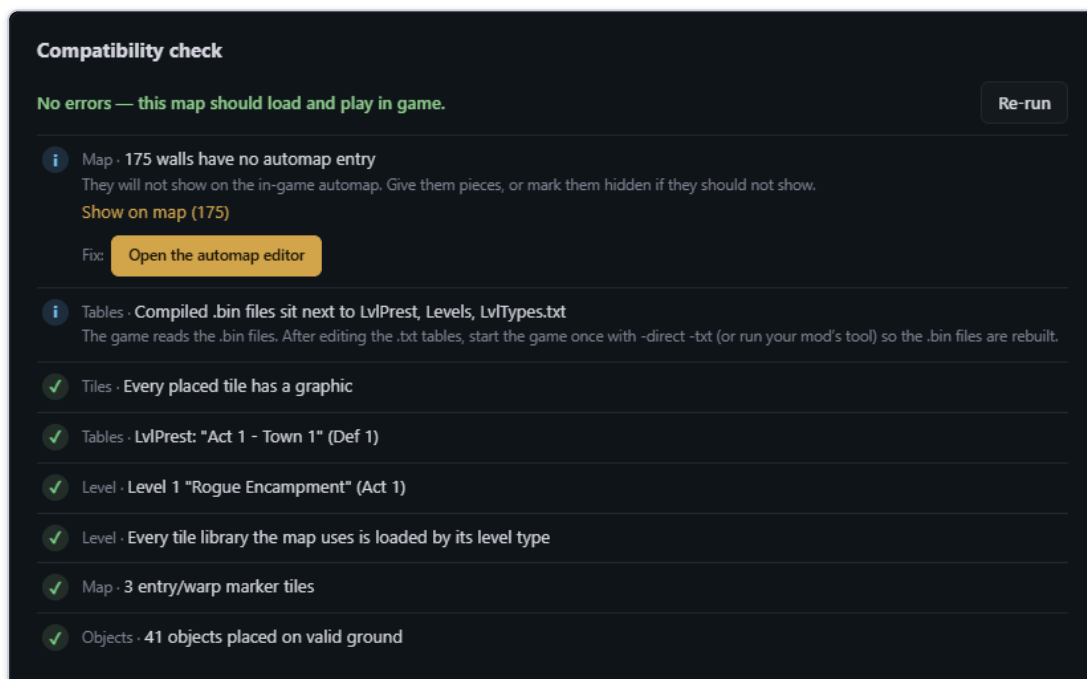
- **Misc.txt:** the map item, with `len` set to the map's level number. PD2's map items open the level named in `len`: "Halls of Torture Map" has `len 193`, which is level 193.
- **patchstring.tbl** (`data/local/lng/eng`): the item's name, and optionally the level's name as shown in game. Every existing string keeps its number.

- **Levels.txt:** the level's `LevelName` / `LevelWarp` point at that name, if you name the level.
- **CubeMain.txt:** the recipe, made the way your mod's other map recipes make their maps (e.g. `,nor` , `lvl` , `ilvl`).

The map must be a level in the game first. If it isn't, the dialog offers [Add to game](#). After creating it, rebuild the `.bin` tables (`-direct -txt`) if your mod uses them.

Recipes and items DS1 Studio made are listed under **Made by DS1 Studio** in the dialog, where you can remove them: one at a time, or tick several (**Select all** ticks every one) and click **Remove selected**. You are asked to confirm first, and the dialog stays open afterwards. Remove an item only when no character holds it: a character with an item whose code is gone may not load; an item can be removed together with the recipes that make it.

9.4 Compatibility check



Game → Compatibility.

Checks the map will load and play in game: missing tiles and DT1s, table wiring, DT1s used but not loaded by the level type, entry markers, a missing waypoint, NPCs on unwalkable ground, stacked objects, walls missing from the automap, compiled `.bin` reminders. Most results offer one-click **fixes**, and **Show on map** marks the cells involved. Map edits made by fixes can be undone.

Ways out. A warp tile whose link has no level in `Levels.txt` leads nowhere; the fix opens the [warp link](#) editor for it. A level built from this map alone, with no working warp and no waypoint, can only be left by town portal (a player without a scroll or tome is stuck): **Add an exit warp...** opens the link editor with the act's town chosen, then gives you the warp tile to click where players should leave. Pick a **Kind of warp** that suits the map before applying. If leaving by town portal is what you want, ignore the warning.

Tile paths too long. The game holds at most 41 characters of a tile path after `data\global\tiles\`; a longer one can crash it. The check lists every such path the level uses, and **Rename...** opens them together: give each a shorter path (a counter shows how many characters are left) and DS1 Studio moves the files, updates every `LvlTypes.txt` and `LvlPrest.txt` row that names them, and keeps the old files and tables as backups.

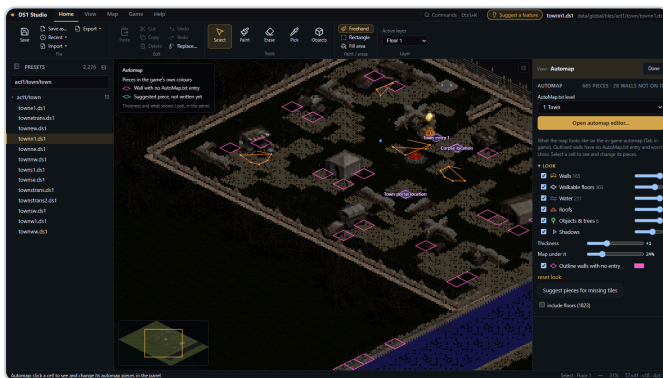
AutoMap rows with no picture. A rule whose Cel1 is `-1` stops the game while it loads AutoMap.txt (earlier versions of **Clear piece** wrote them). The check finds them and removes them in one click, backing AutoMap.txt up first.

Colours. Tile libraries the level loads that weren't drawn for its act and use colours that change between acts are listed, with how much of each; **Convert them to Act 0 colours** fixes them in your mod (originals kept as `.bak`). If the map came from someone working with an Act 0 tile pack (such as Gimli's), their copies of those files are the exact match.

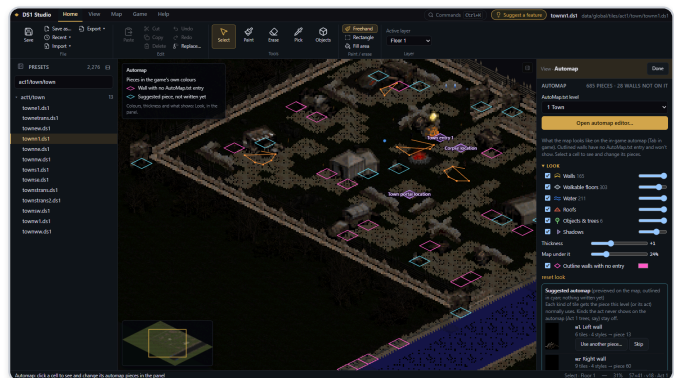
Tiles with no graphic. The check names the tile libraries the map itself lists. If you have them all and tiles are still missing, your copies differ from the ones it was made with (edited versions, or an Act 0 pack): ask the maker for their files. Adding another library that happens to use the same tile numbers, or clearing the cells, loses the maker's tiles.

Maps from someone not using DS1 Studio A map is usually sent as the DS1, the DT1s the maker changed, and rows of their AutoMap.txt. Three things differ between mods: the **DT1s they didn't send** (their copies may be Act 0 or edited: the check above tells), the **level-type numbers** in their AutoMap rows (use **Import** → **AutoMap rows...**: it lists each `LevelName` in the file and adds the rows under the level type you choose, the open map's by default, skipping rows already there), and **files that already exist in your mod** (the DT1 import names the levels that load a file it would replace). When a map import finds two picked files with the same name, the one in a spare folder (`og/`, `old/`, `backup/` ...) is never used over the real one.

9.5 The automap



Automap preview (A).



Suggested pieces previewed in cyan.

View → **Automap** (A) shows the in-game automap over the map. Walls outlined (pink unless you change it) have no AutoMap.txt entry: a hole in the automap. Floors without one are normal (the game draws few floors). Click a cell to see the automap piece of each of its tiles: **Change piece...** picks another (**No piece**, first in the list, leaves those tiles off the automap), and **Clear piece** takes the tile off the automap straight away. Both work on **Only this cell** by default: the cell gets a tile of its own (a copy under a new tile number), so the rest of the level keeps its pieces; choose the wider scope to change every tile of that kind (style and sequence) on the level instead. Clearing removes the matching AutoMap.txt rows (a row with Cel1 `-1` crashes the game while it loads, so DS1 Studio never writes one and refuses to save a table that has one); a cleared tile has no entry, so it is outlined like any tile without a piece. Where a tree or roof stands in a wall's place, the cell has no wall tile and so no wall piece: the cell panel offers **choose a wall piece...** to draw one there anyway.

Look (in the Automap panel, and **Look** on the editor's preview) decides how it is drawn, and is remembered:

- Pieces show in their own `MaxiMap.dc6` colours, as in game. (Colouring each kind of piece its own way is switched off for now.)

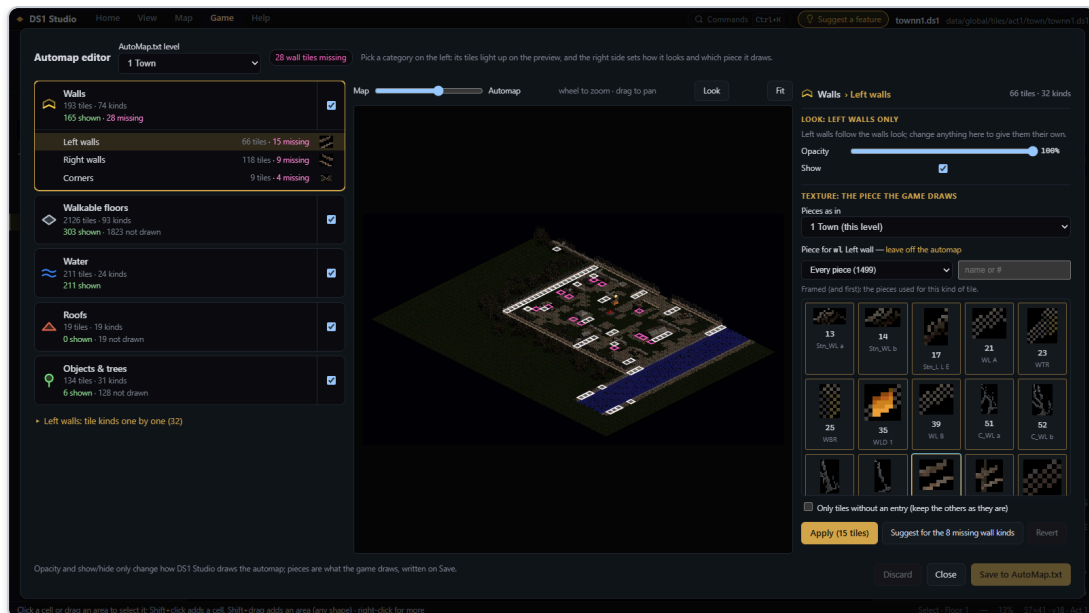
- Each kind can be shown or hidden, with how many pieces of it there are.
- **Opacity, Thickness** (as thin as in game, or thicker to see at any zoom) and **Map under it** (how much the map underneath shows).
- **Outline walls with no entry**, in a colour of your choice. **reset look** goes back to the defaults.

The floating legend on the map follows the look. **Game** → **Automap editor** (or **Open automap editor...** in the panel) sorts the map's tiles into categories: **Walls, Walkable floors, Water, Roofs, Objects & trees** and **Shadows**. Click a category and all its tiles light up on the preview; the right side then sets its **Look** (opacity, show) and its **Texture**, the automap piece the game draws. A selected category opens into its parts: Walls into **Left walls, Right walls, Corners, Doors, Wall ends** and **Lower walls**; Objects into **Trees** and **Columns & props**. Each part shows its tile count, what is missing and the piece it draws now. Click one and the right side works on that part alone: its **Look** (opacity, show) gives it a look of its own (**Use the walls look again** undoes it), and every automap piece is shown below, the ones used for that kind of tile framed and first, to replace just that part's piece (corners keep one piece per corner shape). Pieces are shown in their own `MaxiMap.dc6` colours, as the game draws them; `AutoMap.txt` has no colour column. Walls without an entry are **missing** (holes); other tiles without one are simply **not drawn**. **Apply** sets the chosen pieces, **Suggest for the N missing wall kinds** fills the holes with the pieces the level uses for look-alike tiles, and **Pieces as in** takes a whole wall set from another level. The **Map** ↔ **Automap** slider over the preview fades between the map and its automap. Nothing is written until you **Save to AutoMap.txt**; **tile kinds one by one** still lists every kind for fine edits.

Draw all the same... (above the list) gives a whole kind of tile the same pieces in one go:

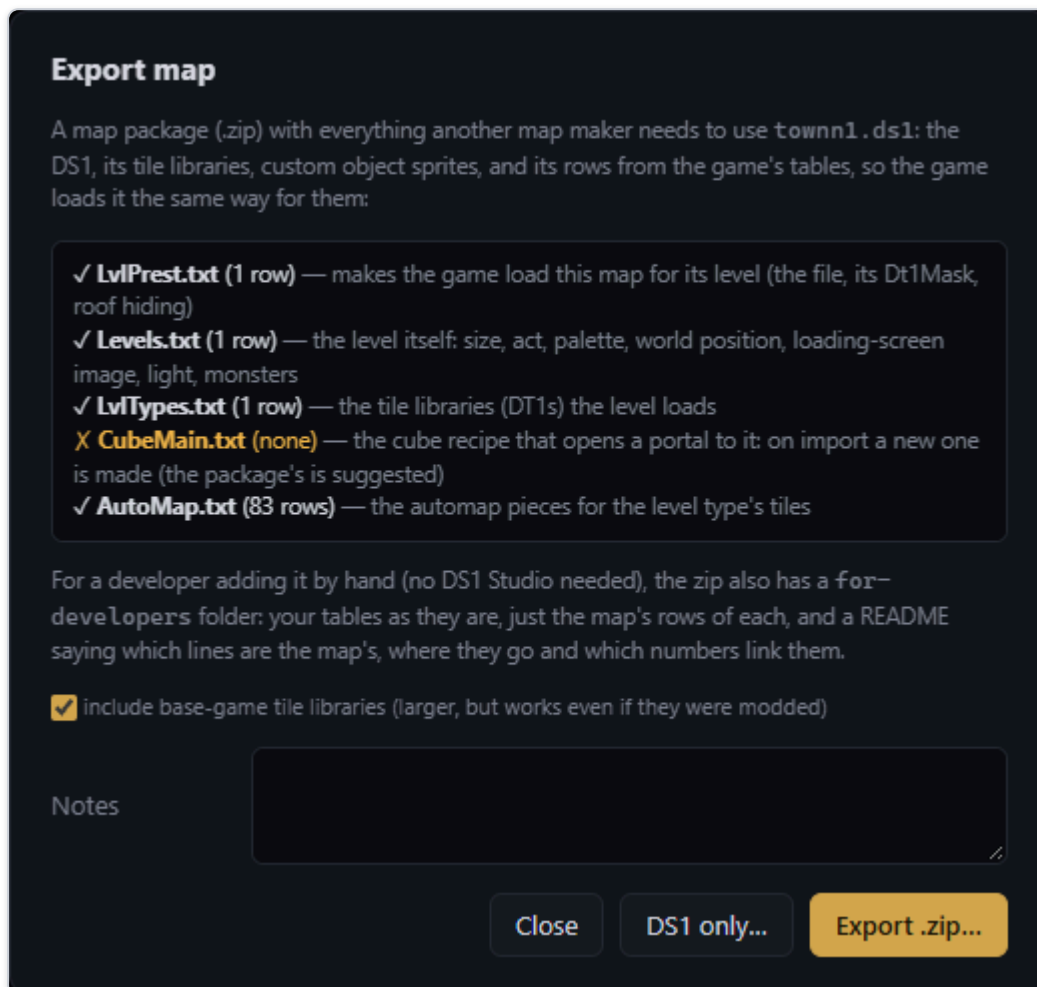
- **Walls**: every wall shape (left and right walls, corners, doors, wall ends, lower walls) gets its own piece, so each still faces the right way, all taken from one wall set — **Pieces as in** any level (this one by default). Click a shape to pick another piece for it.
- **Floors**: one piece for every floor, or leave them all off the automap (as most of the game's levels do).
- **Water**: **Find the water tiles** picks out the floors nobody can walk on that are dark or bluish (rivers and lakes; lava and blocked ground are left out), shows them to untick any that aren't water, and gives them one piece — the level's own water piece to start with.
- **Objects & trees**: one piece each for trees and for columns/props, or left off.
- **Only kinds without an entry** fills the gaps and keeps the rest as it is.

The result goes into the changes shown on the preview; **Save to AutoMap.txt** writes it.



The Automap editor.

10.1 Map packages



Home → File → Export ▾ → Map package.

Export ▾ → Map package... (Home → File) puts the DS1, its DT1s, any modded object sprites and every table row the map needs into one .zip: its **LvlPrest**, **Levels** and **LvlTypes** rows, the **CubeMain** recipe made for it (with its Misc.txt item), the **AutoMap** rows of its level type, and LvlWarp/LvlMaze rows. The dialog lists the five main tables with how many rows each gives, and warns about any the map has none in (not added to the game yet, no cube recipe, no automap pieces). **DS1 only...** saves just the map file.

Import ▾ → Map... with a .zip merges such a zip into your mod safely: same-named rows are reused, clashing ids get new numbers (AutoMap rows follow a renumbered level type), DT1s go into free slots and the **Dt1Mask** is recomputed; AutoMap rows already there aren't added twice. Importing the same map twice changes nothing. The import lists which of the five tables came with the map and warns about missing ones: your own rows are used for those, and the game can crash if they don't match the map.

The maker's **cube recipe and map item aren't copied**: their item code, the level number the item opens and its name only fit the maker's tables. Instead, with **make a cube recipe and map item for it next** (on by default), the imported map opens and the **Cube recipe** tool starts with the maker's ingredients and item name. **Create** makes an item with a free code that opens the map's level number in *your* tables, names it and the level, and refuses a recipe your mod already uses: enter a game, cube it, and use the portal.

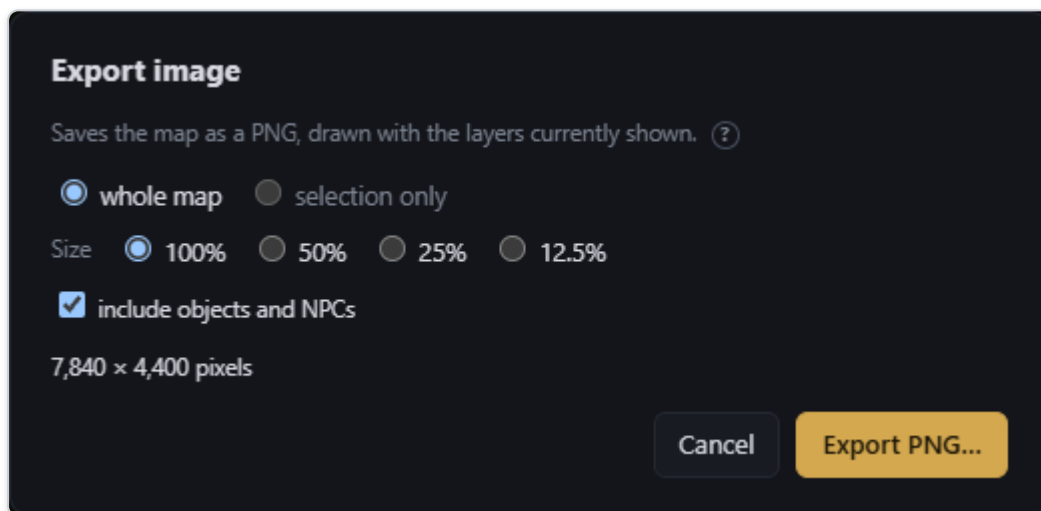
For a developer adding the map by hand

A developer who doesn't use DS1 Studio can add the map with a text editor: every package has a `for-developers` folder with

- `README.txt` : for each table, which lines of the maker's table are the map's (e.g. *Levels.txt line 214, Id 212*), where they go in your own table, and which numbers link the tables, so they can be renumbered safely. Levels, LvlPrest and LvlTypes are read by row position, so their rows are always appended with the next Id/Def; the README says what else to change when an Id changes (LvlPrest `LevelId` , `LevelType` , Misc `len` ...).
- `full/` : the maker's tables exactly as they are, to look the lines up or compare.
- `rows/` : just the map's rows of each table under its header line, ready to paste.
- `strings.txt` : the names the map uses and their text.

DS1 Studio's own import leaves that folder alone.

10.2 Export image



View → Export picture (also Home → File → Export ▾ → Picture).

Saves the whole map or the selection as a PNG at 100%, 50%, 25% or 12.5%, with or without objects, drawn with the layers currently shown. Sizes too large for one image are greyed out.

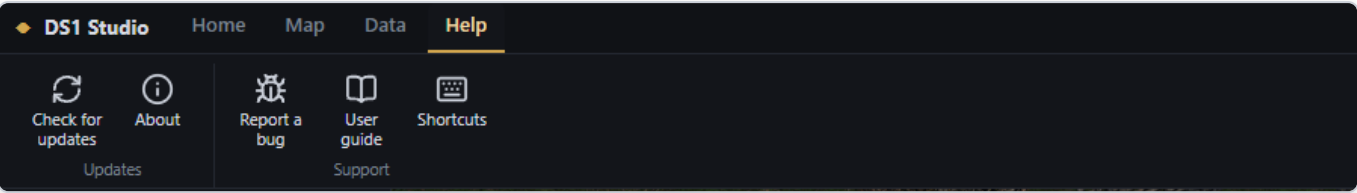
Copy view (Print Screen)

View → **Copy view** (also Home → File → Export ▾), or the **Print Screen** key, copies the map pane exactly as it is on screen (zoom, overlays, selection, minimap) as a picture: paste it anywhere with **Ctrl+V**, into a chat, a document or an image editor. Windows 11 opens its own Snipping Tool on Print Screen unless *Settings* → *Accessibility* → *Keyboard* → *Use the Print screen key to open screen capture* is off; either use the button, turn that off, or give Copy view another key in [Keyboard shortcuts](#).

10.3 Export .ds1

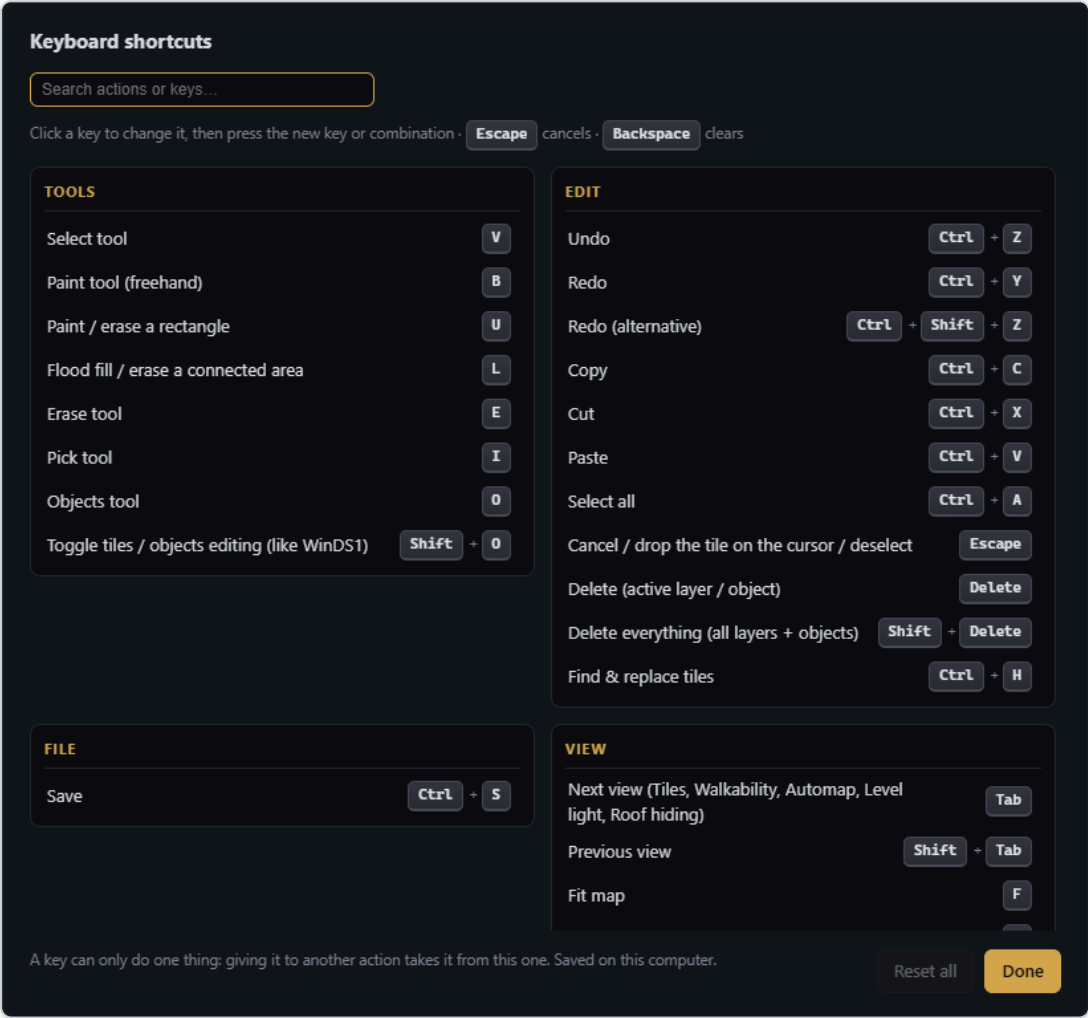
Export map → **DS1 only...** saves a copy of just the map file anywhere on your computer, without touching your mod. It carries no tile libraries or table rows: to hand a map to someone, export the package.

11 Help, feedback and shortcuts



The Help ribbon.

11.1 Keyboard shortcuts



Help → Shortcuts: click a key to change it.

Every shortcut can be changed in **Help** → **Shortcuts**: click a key, then press the new key or combination. The full list is in [Appendix A](#).

11.2 Suggest a feature, report a bug



The top bar.

Suggest a feature (always in the top bar, and in Help) and **Help** → **Report a bug** open a pre-filled GitHub issue with your version, system and the open map. Screenshots help: paste them straight into the issue.

11.3 About

Help → **About** shows the version, build and platform, with links to the GitHub page and release notes.

12 Safety and troubleshooting

[↑ Contents](#)

Before you start DS1 Studio is in beta. Keep a backup of your mod (and your game files) before editing.

12.1 What DS1 Studio writes

- Only into your **mod folder**: maps, DT1s, `.txt` tables, presets. Your Diablo II install is only ever read.
- A file it replaces is kept as `<name>.bak` the first time, so the original survives.
- Autosaved copies live in the app's own storage, not in your folders (1.6).

12.2 If something goes wrong

If part of the app fails, it shows what happened instead of a blank window, with **Try again**, **Close this map**, **Report this bug** (pre-filled with the error details) and **Reload DS1 Studio**. Your unsaved changes are kept in the autosave.

When the game crashes

Diablo II writes every crash to a log in its folder (`D2<date>.txt`). **Game** → **Crash log** reads the newest crashes from your game and mod folders and says, in plain words, what they mean and what to do: a missing loading-screen image (`EntryFile`), a level placed too far out for the automap, `AutoMap 1` on a level that isn't a town, a damaged `.tbl`, or a tile library, map or sprite the game couldn't open just before. The same crash repeated is shown once with a count. When the latest crash is one a map can cause, **Run compatibility check** finds and fixes it. Crashes it doesn't recognise are shown with where they happened, for reporting.

12.3 Compiled .bin tables

The game reads compiled `.bin` copies of the `.txt` tables. After changing tables (Add to game, warps, tile libraries, cube recipes...), start the game once with `-direct -txt` so it rebuilds them, if your mod ships `.bin` files.

12.4 Common questions

Question	Answer
Tiles show as red “missing” diamonds.	The map uses tiles no loaded DT1 provides. Run the Compatibility check — it offers to add the DT1s that contain them — or add them in Tile libraries .
My new map doesn't appear in game.	Use Add to game , connect it with a warp or a cube recipe , and rebuild <code>.bin</code> tables (12.3).
A warp leads nowhere.	Its link isn't set in Levels.txt: select it and Connect it to a level... (chapter 6).
Edited tiles look different in another act.	Use the Act 0 palette and Make act-safe in the DT1 editor .
Windows blocks the installer.	More info → Run anyway (1.1).

A Keyboard and mouse reference

[↑ Contents](#)

Default keys; change them in [Help → Shortcuts](#).

Tools

Select tool	V
Paint tool (freehand)	B
Paint / erase a rectangle	U
Flood fill / erase a connected area	L
Erase tool	E
Pick tool	I
Objects tool	O
Toggle tiles / objects editing	Shift + O

File and edit

Save	Ctrl + S
Undo / redo	Ctrl + Z / Ctrl + Y (Ctrl + Shift + Z)
Copy / cut / paste	Ctrl + C / X / V
Select all	Ctrl + A
Cancel / drop the tile on the cursor / deselect	Esc
Delete (active layer / object)	Delete
Delete everything (all layers + objects)	Shift + Delete
Find & replace tiles	Ctrl + H

View

Next / previous view	Tab / Shift + Tab
Fit map	F
Game view	Z
Grid	G
Rooms (8×8)	R
Walkability	W
Automap preview	A
Object markers / sprites / NPC paths	M / N / P
Minimap	K
Roof hide areas / as if inside	H / Shift + H

Layers (show / hide)

Floor 1 / Floor 2	1 / 2
Wall 1 – Wall 4	3 – 6
Shadows / roofs / lower walls	7 / 8 / 9
Special tiles	0

Mouse

Pan the map	Arrow keys (Shift = faster), Space + drag, middle or right drag
Zoom	Mouse wheel
Step through stacked tiles	Shift + wheel
Stack a paste / preset onto existing tiles	Alt + click
Add a tile to the random paint mix	Ctrl + click it in the Tiles panel
Pin / unpin a tile	Right-click it in the Tiles panel

Zoom tile / object thumbnails	Ctrl + wheel over the panel
Move the view	Click or drag on the minimap
Select a range of tiles (DT1 editor)	Shift + click

Pixel painter (DT1 editor)

B pencil · **E** eraser · **G** fill · **I** colour picker · **[** / **]** brush size · **Alt** + click picks a colour · **Ctrl** + **Z** / **Ctrl** + **Y** undo / redo · **Ctrl** + wheel zoom (8.4).

DS1	A map preset: a grid of cells with floor, wall, shadow and tag layers, plus objects and NPCs. The game builds levels from them.
DT1	A tile library: the pictures and settings of the tiles DS1 cells refer to (chapter 8).
Cell	One diamond of the map grid; each layer holds at most one tile in it.
Main / sub index	A tile's number. A cell asks for "kind + main + sub" and the game draws the DT1 tile with those numbers.
Orientation (kind)	What a tile is: floor, left/right wall, corner, door, pillar/tree, roof, shadow, lower wall, special tile.
Variant / rarity	Several tiles can share a number; the game picks one at random, weighted by rarity. Animated tiles use it as their frame number.
Sub-tile flags	The 5×5 walkability grid of a tile: block walk, sight, missiles, teleport... (3.8).
Special tile	An invisible marker: warps, entries, corpse and portal spots (2.7).
Level	A row of <code>Levels.txt</code> : one area of the game (a town, a dungeon floor), built from DS1 presets or at random.
LvlPrest.txt	Lists the DS1 files, the level each belongs to, and its Dt1Mask .
LvlTypes.txt	Level types: up to 32 DT1 <code>File</code> slots each.
Dt1Mask	Which of its level type's DT1 slots a DS1 loads (one bit per slot).
Warp / link	How levels connect: <code>Levels.txt</code> <code>Vis0–7</code> (target level) and <code>Warp0–7</code> (LvlWarp row) (chapter 6).
Preset (DS1 Studio)	A saved piece of map you can place again (chapter 5). Not to be confused with a DS1 "map preset".
Act 0 palette	The colours that look the same in every act (8.2).
Mod folder	Your mod's folder; everything DS1 Studio saves goes here (1.5).

Diablo II and its data are © Blizzard Entertainment. DS1 Studio reads your own game files and never ships them. Thanks to Paul Siramy (WinDS1, DT1 Tools), the Phrozen Keep community, and Gimli for the Act 0 palette.

[↑ Back to the start](#)